

Міністерство освіти і науки України
Український державний університет науки і технологій

Навчально-науковий інститут
«Український державний хіміко-технологічний університет»

(назва навчально-наукового інституту)

Факультет комп'ютерних наук та інженерії

(повна назва факультету)

Кафедра інформаційних систем

(повна назва кафедри)

Пояснювальна записка

до дипломної роботи

бакалавра

(освітній рівень)

на тему Розробка та програмна реалізація алгоритму оптимізації
розподілу вантажів у транспортній мережі

Виконав: студент 4 курсу, групи КН-22
спеціальності

122 Комп'ютерні науки

(код і назва спеціальності)

Ніколаєв Т.О.

(прізвище та ініціали)

(підпис)

Керівник

Зеленцов. Д.Г.

(прізвище та ініціали)

(підпис)

Рецензент

Коротка Л.І.

(прізвище та ініціали)

(підпис)

Дніпро – 20 26 року

Український державний університет науки і технологій
(повне найменування вищого навчального закладу)

Навчально-науковий інститут
«Український державний хіміко-технологічний університет»
(назва навчально-наукового інституту)

Факультет, відділення	комп'ютерних наук та інженерії, денне
Кафедра	інформаційних систем
Освітній рівень	бакалавр
Спеціальність	122 Комп'ютерні науки (код і назва)
Освітня програма	Комп'ютерні науки. Штучний інтелект (назва)

ЗАТВЕРДЖУЮ

Завідувач кафедри _____
канд. техн. наук, доц. Ляшенко О.А.
« ____ » _____ 20 ____ року

З А В Д А Н Н Я
НА ДИПЛОМНУ РОБОТУ СТУДЕНТУ
Ніколаєву Тимуру Олександровичу
(прізвище, ім'я, по-батькові)

1. Тема роботи Розробка та програмна реалізація алгоритму оптимізації розподілу вантажів у транспортній мережі
- керівник роботи Зеленцов Дмитро Гегемонович, д-р. техн. наук, професор,
(прізвище, ім'я, по-батькові, науковий ступінь, вчене звання)
- затверджені наказом вищого навчального закладу від 02.03.2026 р. № 77-к
2. Строк подання студентом роботи 05.06.2026 р.
3. Вихідні дані до роботи Картографічні дані, мультимодальна транспортна мережа, обмеження вантажомісткості транспортних засобів
4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) _____
Загальний розділ.
Спеціальний розділ.
Розрахунковий розділ.
5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)
Презентація до дипломної роботи.

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв
1 Загальний розділ	Зеленцов Д. Г., професор		
2 Спеціальний розділ	Зеленцов Д. Г., професор		
3 Розрахунковий розділ	Зеленцов Д. Г., професор		
Нормоконтроль	Зеленцов Д. Г., професор		

7. Дата видачі завдання 11.05.2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів дипломної роботи	Строк виконання етапів роботи	Примітка
1	Розробка програмного продукту	11.05.2026-21.05.2026	виконано
2	Оформлення пояснювальної записки	11.05.2026-21.05.2026	виконано
3	Попередній захист дипломної роботи	25.05.2026	виконано
4	Подання дипломної роботи на перевірку та підпис керівнику, проходження нормоконтролю	21.05.2026-26.05.2026	виконано
5	Перевірка дипломної роботи на академічний плагіат	26.05.2026-01.06.2026	виконано
6	Підготовка презентації та доповіді	26.05.2025-01.06.2026	виконано
7	Рецензування дипломної роботи	02.06.2026-04.06.2026	виконано
8	Подання дипломної роботи на підпис завідувачу кафедри	04.06.2026	виконано
9	Захист дипломної роботи	10.06.2026	

Студент

(підпис)

Ніколаєв Т.О.

(прізвище та ініціали)

Керівник роботи

(підпис)

Зеленцов Д.Г.

(прізвище та ініціали)

РЕФЕРАТ

Пояснювальна записка до дипломної роботи: 74 с., 3 табл., 17 рис., 2 дод., 12 джерел.

ТРАНСПОРТНА МЕРЕЖА, МУЛЬТИМОДАЛЬНИЙ ГРАФ, МАРШРУТИЗАЦІЯ ВАНТАЖІВ, ГЕНЕТИЧНІ АЛГОРИТМИ.

Об'єкт роботи – проблема розподілу та доставки вантажів у мультимодальній транспортній мережі.

Предмет роботи – математичні моделі та еволюційні алгоритми в задачах маршрутизації транспорту.

Мета роботи – розробка та програмна реалізація алгоритму оптимізації маршрутів доставки для мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів.

Методи дослідження: теорія графів, математичне моделювання, еволюційні методи.

Засоби розробки – середовище розробки Visual Studio Code та мова програмування Python.

Структура роботи включає три розділи. Загальний розділ звіту містить обґрунтування актуальності розробки, аналіз сучасних підходів до маршрутизації, опис структури транспортної мережі, аналіз аналогів програмних продуктів, аналіз літератури та постановку задачі.

У спеціальному розділі виконано математичну постановку задачі: мережу формалізовано у вигляді графу з атрибутами вузлів та ребер, враховано обмеження за вантажною місткістю; описано генетичний алгоритм.

В розрахунковому розділі представлено інформацію про програмне та технічне забезпечення, опис програми, інструкція користувача та програміста, результати тестування системи.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	7
ВСТУП.....	8
1 ЗАГАЛЬНИЙ РОЗДІЛ	10
1.1 Обґрунтування актуальності дослідження.....	10
1.2 Опис предметної області та аналіз об’єкта дослідження	10
1.3 Огляд і аналіз літератури за темою роботи	12
1.4 Математична постановка задачі	13
1.5 Висновки за загальним розділом	14
2 СПЕЦІАЛЬНИЙ РОЗДІЛ.....	15
2.1 Проєктування розроблюваної інформаційної системи.....	15
2.1.1 Функціональна модель системи (діаграма прецедентів).....	17
2.1.2 Структурна модель – діаграма класів предметної області	19
2.1.3 Структурна модель – модульна архітектура	21
2.1.4 Поведінкова модель – геокодування адреси.....	25
2.1.5 Поведінкова модель – оптимізація одного транспортного засобу	28
2.1.6 Поведінкова модель – оптимізація багатьох ТЗ	31
2.1.7 Динамічна модель – діаграма діяльності	35
2.2 Інформаційне забезпечення	39
2.2.1 Концептуальний рівень – сутності предметної області.....	39
2.2.2 Логічний рівень – реляційна схема та потоки даних	43
2.2.3 Фізичний рівень – реалізація у СКБД SQLite	46
2.3 Алгоритмічне забезпечення	48

	6
2.4 Висновки за спеціальним розділом	50
3 РОЗРАХУНКОВИЙ РОЗДІЛ	51
3.1 Програмне забезпечення	51
3.2 Технічне забезпечення	52
3.3 Опис програми.....	53
3.4 Інструкція користувача.....	56
3.5 Інструкція програміста.....	59
3.6 Тестування розробленого програмного забезпечення	60
3.7 Результати чисельних експериментів	65
3.8 Висновки за розрахунковим розділом	68
ВИСНОВКИ	70
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ.....	74
ДОДАТОК А	75
ДОДАТОК Б.....	11

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

ТЗ – транспортний засіб

ПЗ – програмне забезпечення

UML – Unified Modeling Language

VRP – Vehicle Routing Problem

VRPTW – Vehicle Routing Problem with Time Windows

CVRPTW – Capacitated Vehicle Routing Problem with Time Windows

OSM – Open Street Map

GA – Genetic Algorithm

DRY – Don't Repeat Yourself

ООП – Об'єктно-орієнтоване програмування

SRP – Single Responsibility Principle

MTZ – Miller-Tucker-Zemlin

TSP – Traveling salesman problem

ВСТУП

Сучасні реалії глобальної та локальної економіки характеризуються стрімким зростанням обсягів вантажних перевезень, що вимагає впровадження високоефективних систем управління логістикою.

У щільній міській забудові «остання миля» з'їдає основну частину логістичного бюджету: автомобільний доступ обмежений пішохідними зонами та одностороннім рухом, а хаби приймають вантаж лише у вузькі часові вікна. Тому при плануванні маршрутів доводиться комбінувати автомобілі, велосипеди та пішохідних кур'єрів – одного транспортного режиму на все місто не вистачає.

Традиційні конструктивні методи планування маршрутів часто не забезпечують знаходження глобального оптимуму в динамічному логістичному середовищі через схильність до застрягання в локальних екстремумах. Це зумовлює актуальність розробки та програмної реалізації спеціалізованих систем на основі генетичних алгоритмів – популяційних метаевристик, які дозволяють ефективно досліджувати простір розв'язків NP-складної задачі маршрутизації та знаходити якісні результати для автоматизації процесів розподілу вантажів.

Об'єкт роботи – проблема розподілу та доставки вантажів у мультимодальній транспортній мережі.

Предмет роботи – математичні моделі та еволюційні алгоритми в задачах маршрутизації транспорту.

Мета роботи – розробка та програмна реалізація алгоритму оптимізації маршрутів доставки для мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів.

Методи дослідження. Для досягнення поставленої мети використано методи теорії графів для формалізації транспортної мережі, методи математичного моделювання для побудови моделі задачі змішаного цілочисельного програмування з урахуванням обмежень вантажної місткості та часових вікон та генетичні алгоритми для знаходження наближених до

оптимальних розв'язків шляхом еволюційного моделювання процесів селекції, схрещування та мутації.

Поставлено такі завдання:

- вивчити предметну область, структуру та принципи функціонування об'єкта дослідження;
- провести аналіз сучасних підходів та аналогів програмних продуктів для вирішення задач маршрутизації;
- формалізувати транспортну мережу як мультимодальний зважений орієнтований граф;
- розробити математичну модель задачі оптимізації з урахуванням обмежень місткості;
- реалізувати програмний модуль на основі розробленої математичної моделі оптимізації та провести його тестування на реальних даних.

Практичне значення одержаних результатів. Розроблене програмне забезпечення та алгоритмічні рішення можуть бути використані транспортно-логістичними компаніями для автоматизованої побудови маршрутів, що дозволить скоротити експлуатаційні витрати та підвищити якість обслуговування клієнтів.

Дана робота брала участь у I турі Всеукраїнського конкурсу студентських наукових робіт з галузей знань і спеціальностей у 2025-2026 н.р та посіла 2 призове місце (Додаток Б).

1 ЗАГАЛЬНИЙ РОЗДІЛ

1.1 Обґрунтування актуальності дослідження

В умовах глобальної цифровізації економіки та стрімкого зростання ринку електронної комерції, оптимізація логістичних процесів стає значним фактором для конкурентоспроможності підприємств. Задача маршрутизації транспортних засобів (Vehicle Routing Problem, VRP) належить до класу NP-складних задач комбінаторної оптимізації [1], що означає неможливість отримання точного розв'язку за прийнятний час для реальних масштабів міської інфраструктури.

Актуальність дослідження зумовлена наступними факторами:

- Економічна ефективність: витрати на логістику можуть складати до 17% від загальної вартості доставки [2]. Навіть мінімальне скорочення маршруту призводить до значної економії палива та людських ресурсів.

- Динамічність середовища: сучасні системи повинні враховувати часові вікна (VRPTW), затори та обмеження вантажності в режимі реального часу.

- Технологічне покращення: класичні евристики (наприклад, алгоритм Кларка-Райта) часто «застрягають» у локальних оптимумах [3]. Використання методів штучного інтелекту, зокрема генетичних алгоритмів, дозволяє ефективно досліджувати простір розв'язків та знаходити результати, близькі до глобального оптимуму, що є пріоритетним завданням для розробки сучасного логістичного ПЗ.

1.2 Опис предметної області та аналіз об'єкта дослідження

Предметна область роботи – планування міських маршрутів доставки для парку з кількох транспортних засобів різних типів. Завдання має три тісно пов'язані рівні. Перший, розподіл: які саме замовлення дістануться машині, які –

велосипеду, а які кур'єру пішки, з огляду на вантажомісткість і фізичну досяжність адреси. Другий, впорядкування: у якій послідовності кожен кур'єр об'їжджатиме свої зупинки. Третій, часові обмеження: жоден маршрут не повинен порушити вікно роботи хабу або клієнта. Якість розв'язку оцінюється сумарним часом руху і кількістю задіяних машин – це ті дві величини, що безпосередньо переходять у витрати на паливо й оплачуваний час персоналу.

Об'єктом дослідження у роботі виступає проблема розподілу та доставки вантажів у мультимодальній транспортній мережі.

Предметом дослідження є математичні моделі та генетичні алгоритми маршрутизації транспортних засобів.

Структурно об'єкт представляє собою сукупність логістичних хабів, пунктів доставки та транспортних коридорів, що їх з'єднують. З математичної точки зору таку структуру доцільно розглядати як мультимодальний зважений орієнтований граф. Визначення графу наведено в формулі (1.1):

$$G = (V, E, \Phi, \Psi) \quad (1.1)$$

де V – вершини графа;

E – ребра графа.

Функціональні особливості об'єкта полягають у динамічній взаємодії різних типів транспортних засобів (автомобілів, велосипедів, кур'єрів-пішоходів) для виконання завдань доставки. Кожна одиниця транспорту має обмежену вантажну місткість, а кожен маршрут повинен бути завершений у межах визначеного часового вікна хабу [4]. Особливе значення має функція «останньої милі», яка передбачає можливість долання частини шляху пішки, якщо прямий доступ транспорту до точки призначення обмежений або неможливий.

1.3 Огляд і аналіз літератури за темою роботи

Проблема маршрутизації транспортних засобів (Vehicle Routing Problem, VRP) є однією з найбільш інтенсивно досліджуваних тем у сучасній комбінаторній оптимізації та дослідженні операцій. Фундаментальні основи математичного опису подібних задач були закладені ще в середині XX століття. Зокрема, для побудови математичних моделей у вигляді задач змішаного цілочисельного програмування критично важливими залишаються методи, запропоновані Міллером, Такером та Земліним [5]. Їхня робота 1960 року впровадила так звані обмеження MTZ (Miller-Tucker-Zemlin) для елімінації підциклів у маршрутах [6]. Це дозволило формалізувати задачу комівояжера та VRP як цілісну систему лінійних обмежень, що стала базовою для всіх наступних розробок у галузі цілочисельного програмування.

Особливе місце в сучасній логістиці посідає задача маршрутизації з часовими вікнами (Vehicle Routing Problem with Time Windows, VRPTW). Класичними у цьому напрямку є праці Маріуса Соломона, який у 1987 році не лише розробив сімейство евристичних алгоритмів (зокрема евристику послідовних вставок I1), а й створив набір тестових задач (Solomon benchmarks), що донині є світовим стандартом для оцінки ефективності алгоритмів [7]. Його підхід базується на критеріях мінімізації додаткового часу та відстані при вставці нового вузла в існуючий маршрут, що дозволяє знаходити допустимі розв'язки для NP-складних задач великої розмірності за поліноміальний час.

Сучасні дослідження значно розширили можливості моделювання реальної інфраструктури. У фундаментальній монографії Паоло Тота та Даніеле Віго «The Vehicle Routing Problem» проведено системний аналіз точних та наближених методів розв'язання VRP [8]. Автори детально розглядають використання теорії графів для побудови мультимодальних моделей, де вузли та ребра характеризуються наборами динамічних атрибутів. Їхні дослідження алгоритмів «гілок та меж» (branch-and-bound) та «гілок та розрізів» (branch-and-

cut) заклали підґрунтя для сучасного математичного забезпечення логістичних систем.

Детальний аналіз метаевристичних підходів був проведений Брейсі та Жендро, які систематизували методи локального пошуку та еволюційні стратегії для VRPTW. Вони довели, що для динамічних міських мереж використання лише конструктивних евристик є недостатнім через їхню схильність до застрягання в локальних оптимумах [9]. У свою чергу, дослідження Ванга та Копфера підкреслюють важливість інтеграції складних процесів у єдині моделі, що враховують не лише магістральні перевезення, а й специфіку доставки «останньої милі» та спільне використання ресурсів декількома логістичними операторами [10].

Аналіз літературних джерел підтверджує, що актуальним завданням є створення алгоритмічного забезпечення, яке б ефективно поєднувало швидкість класичних евристик Соломона (для генерації початкової популяції) із потужним глобальним пошуком генетичних алгоритмів у межах мультимодальних графових моделей.

1.4 Математична постановка задачі

Математична модель задачі базується на теорії графів та методах змішаного цілочисельного програмування.

За формулою (1.1), мережа формалізується як орієнтований граф.

Як цільова функція розглядається функція мінімізації сумарних витрат, що складаються з часу переїздів та кількості задіяних транспортних засобів. Визначення цільової функції наведено в формулі (1.2).

$$F = A \cdot \sum_{k \in V} \sum_{i,j \in N} t_{ij} x_{ij}^k + B \cdot \sum_{k \in V} y^k \rightarrow \min \quad (1.2)$$

де t_{ij} – час у дорозі між вузлами i та j , який залежить від відстані та швидкості транспортного засобу;

x_{ij}^k – бінарна змінна (1, якщо ТЗ k їде від i до j , інакше – 0);

y^k – бінарна змінна використання транспортного засобу k ;

A, B – вагові коефіцієнти пріоритетності (час та кількість транспорту).

Обмеження:

- Місткість: сумарна вага вантажів у кожному ТЗ не повинна перевищувати його ліміт Q ;
- Часові вікна: прибуття у вузол i має відбутися в межах $[e_i, l_i]$;
- Зв'язність: кожний клієнт повинен бути відвіданий рівно один раз.

1.5 Висновки за загальним розділом

У результаті аналізу предметної області встановлено, що задача маршрутизації (VRP) у сучасних мультимодальних мережах є NP-складною, що унеможливорює використання виключно точних методів для реальних масштабів інфраструктури. Огляд літератури та існуючих підходів підтвердив, що класичні конструктивні евристики часто застрягають у локальних оптимумах, тому виникає гостра потреба у використанні метаевристичних методів. Доведено актуальність розробки алгоритмічного забезпечення на основі генетичних алгоритмів, які здатні ефективно знаходити наближені до глобального оптимуму розв'язки з урахуванням жорстких часових вікон та специфіки «останньої милі». Сформована теоретична база та математична модель об'єкта стають основою для подальшої програмної реалізації логістичної системи.

2 СПЕЦІАЛЬНИЙ РОЗДІЛ

2.1 Проєктування розроблюваної інформаційної системи

Для проєктування системи оптимізації розподілу вантажів у мультимодальній мережі обрано об'єктно-орієнтований підхід (ООП), що забезпечує гнучкість при моделюванні складних логістичних сутностей та природне відображення предметної області у програмний код. Вибір ООП-підходу зумовлений рядом міркувань: він узгоджується з принципом концептуальної спільності, який вимагає використання однієї парадигми як на етапі аналізу й проєктування, так і на етапі реалізації; також обрана мова програмування Python надає повноцінну підтримку об'єктно-орієнтованої парадигми через класи, успадкування, поліморфізм та інкапсуляцію; логістичні сутності (транспортний засіб, точка доставки, маршрут, пакунок) природно відображаються в об'єкти зі станом і поведінкою, що робить ООП-декомпозицію інтуїтивно зрозумілою. Декомпозиція системи виконана за допомогою CASE-засобу PlantUML, що дозволяє формалізувати структуру системи через стандарти UML.

Як інструмент моделювання застосовано CASE-засіб PlantUML – текстовий генератор UML-діаграм, який дозволяє описувати моделі мовою спеціалізованої DSL та автоматично формувати графічне представлення. Перевагами PlantUML є: текстовий формат вихідних файлів (придатний для версійного контролю в Git та порівняння змін у diff-режимі), повна підтримка специфікації UML 2.5, незалежність від графічних редакторів та можливість швидкого внесення змін до моделей без ручного перерисовування. Усі побудовані діаграми відповідають нотації UML 2.5 та сформовані у вигляді векторних PNG-зображень з єдиними стильовими параметрами.

Проєктування системи виконано у чотирьох взаємодоповнюючих аспектах моделювання, які разом утворюють повну архітектурну специфікацію інформаційного продукту:

– Функціональний аспект – описує систему з точки зору її користувачів та зовнішніх сервісів, формалізує сукупність бізнес-сценаріїв і визначає функціональну межу продукту. Представлений діаграмою прецедентів (Use Case diagram) – п. 2.1.1.

– Структурний аспект – фіксує статичну будову системи: сутності предметної області, їх атрибути та відношення, а також модульну декомпозицію вихідного коду. Представлений двома діаграмами класів: предметної області (п. 2.1.2) та модульної архітектури (п. 2.1.3).

– Поведінковий аспект – деталізує взаємодію компонентів у часі для конкретних бізнес-сценаріїв. Представлений трьома діаграмами послідовностей (sequence diagrams), що покривають ключові динамічні процеси системи: геокодування адреси (п. 2.1.4), одно-транспортна оптимізація маршруту (п. 2.1.5) та багатотранспортна оптимізація VRP (п. 2.1.6).

– Динамічний аспект – узагальнює потік управління та обробки даних у вигляді єдиного бізнес-процесу з явним зображенням точок прийняття рішень, паралельних потоків та сценаріїв обробки помилок. Представлений діаграмою діяльності (activity diagram) – п. 2.1.7.

Загалом для опису архітектури системи побудовано сім UML-діаграм, кожна з яких розглядає продукт під своїм кутом зору та доповнює інші, утворюючи повну й внутрішньо узгоджену модель. Послідовність викладу обрано від найбільш абстрактних і користувацько-орієнтованих моделей (функціональна) через статичну структуру (структурні) до динамічних аспектів виконання (поведінкові, діяльнісна). Така послідовність відповідає принципам стандарту ISO 42010 щодо опису архітектури програмного забезпечення і дозволяє читачеві поступово занурюватися у деталі реалізації, починаючи з загальної функціональної картини [11].

2.1.1 Функціональна модель системи (діаграма прецедентів)

Для формалізації функціональних вимог до інформаційної системи та визначення меж її відповідальності побудовано діаграму прецедентів (Use Case diagram) у стандарті UML 2.5. Діаграма зображує всю сукупність функціональних можливостей системи з точки зору акторів, які з нею взаємодіють, і дозволяє чітко окреслити функціональну межу розроблюваного програмного продукту. Діаграму прецедентів наведено на рисунку 2.1.

На діаграмі виділено чотири актори. Первинним є Оператор логістики – користувач системи, який безпосередньо ініціює всі бізнес-сценарії через веб-інтерфейс. Решта три актори є вторинними (системними) і відповідають зовнішнім інформаційним сервісам, з якими взаємодіє розроблюваний продукт: Nominatim API – сервіс прямого та зворотного геокодування адрес у складі проєкту OpenStreetMap; OpenStreetMap (Overpass API) – джерело первинних картографічних даних про вуличну мережу; Mapbox API – комерційний сервіс отримання матриці часів руху з урахуванням реального трафіку (використовується опціонально).

Функціональність системи представлено сукупністю прецедентів, які можна об'єднати у чотири логічні групи. До групи керування вхідними даними належать прецеденти «Обрати місто доставки», «Вказати адресу депо», «Додати адресу доставки», «Додати точку на карті» та «Видалити точку доставки» – вони забезпечують формування первинного набору точок доставки. Група конфігурації автопарку представлена прецедентом «Керувати парком ТЗ», що дозволяє додавати, редагувати та видаляти транспортні засоби з заданням їх режиму пересування і вантажомісткості. Група розрахункових операцій містить ключовий прецедент «Запустити оптимізацію», який є точкою входу для повного циклу обчислень. Група візуалізації та обліку – це прецеденти «Переглянути результати на карті», «Переглянути таблицю маршруту» та «Керувати пакунками», які забезпечують подання результатів та облік стану замовлень.

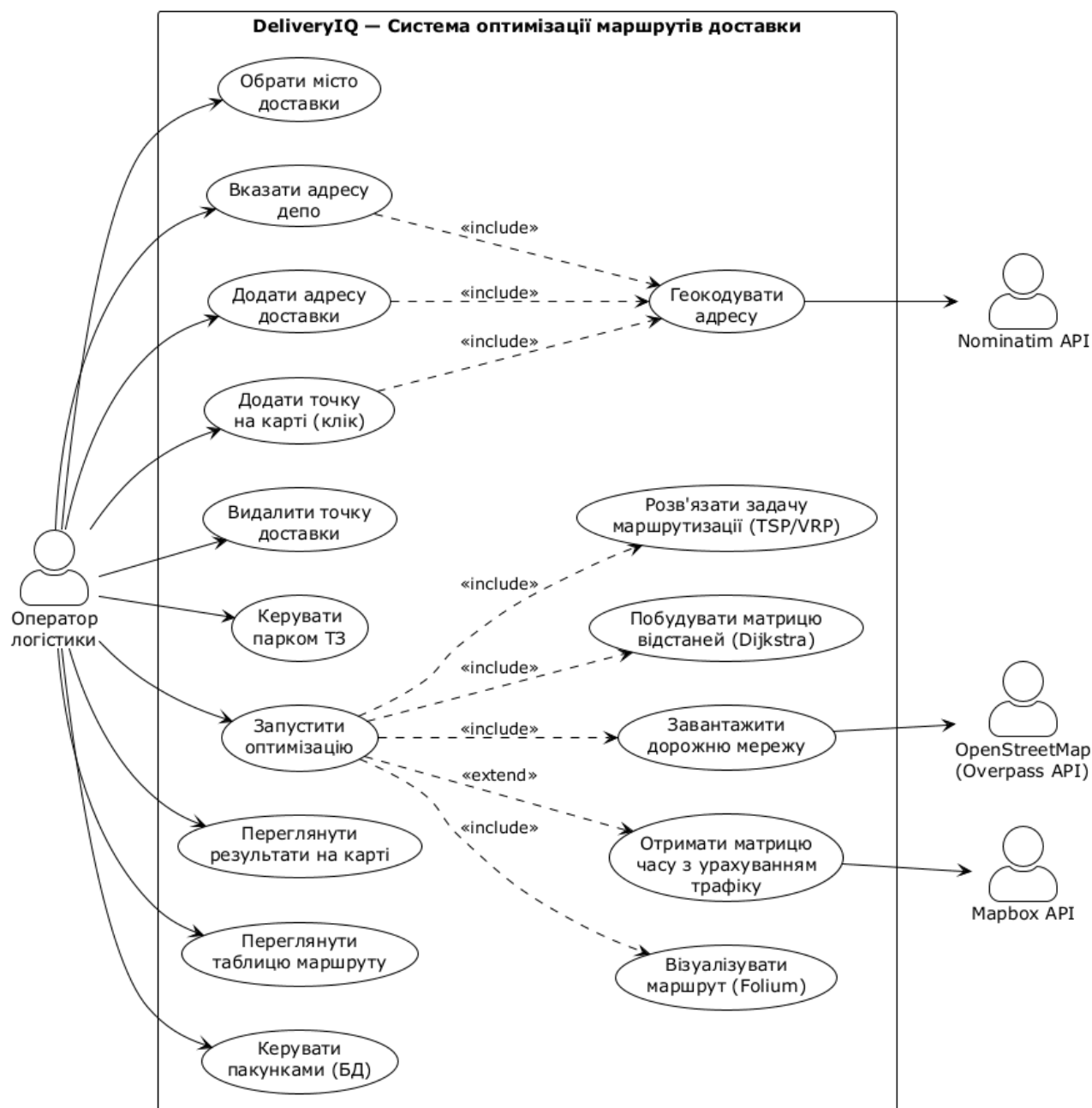


Рисунок 2.1 – Діаграма прецедентів

Між прецедентами встановлено два типи відношень, передбачених стандартом UML. Відношення «include» (включення) використано для декомпозиції складених прецедентів на обов'язкові підпроцеси. Так, прецедент «Запустити оптимізацію» обов'язково включає підпроцеси «Завантажити дорожню мережу», «Побудувати матрицю відстаней (Dijkstra)», «Розв'язати задачу маршрутизації (TSP/VRP)» та «Візуалізувати маршрут (Folium)». Аналогічно прецеденти введення адрес («Вказати адресу депо», «Додати адресу

доставки», «Додати точку на карті») включають прецедент «Геокодувати адресу», оскільки без перетворення текстової адреси у пару координат подальша обробка неможлива. Відношення «extend» (розширення) застосовано до прецеденту «Отримати матрицю часу з урахуванням трафіку», який розширює базовий прецедент оптимізації лише за наявності у конфігурації валідного ключа Mapbox API; у разі його відсутності система прозоро повертається до використання статичної матриці часів, побудованої на основі OSM-графа.

2.1.2 Структурна модель – діаграма класів предметної області

Для опису внутрішньої структури інформаційної системи з точки зору сутностей предметної області та відношень між ними побудовано діаграму класів предметної області. На відміну від модульної діаграми класів, що відображає програмну архітектуру (див. п. 2.1.3), діаграма предметної області абстрагується від реалізаційних деталей і фокусується на бізнес-сутностях логістичної системи та їх атрибутах. Діаграму класів предметної області наведено на рисунку 2.2.

Всі класи предметної області згруповано у п'ять логічних пакетів, кожен з яких відповідає окремому аспекту функціонування системи.

Пакет «Доставка» містить три ключові класи. Клас `DeliveryStop` є центральною сутністю системи й описує одну точку доставки: атрибути `address` (адреса), `lat/lon` (координати), `node_id` (індекс вузла на дорожньому графі), `weight_kg` (вага вантажу) та `source` (джерело: введення вручну або клік на карту). Клас `LegInfo` представляє один сегмент маршруту з атрибутами `from_label`, `to_label` (мітки початкового та кінцевого пунктів), `distance_m`, `travel_time_s` та `cumulative_time_s` (загальний час). Клас `ModeResult` агрегує сукупність сегментів `LegInfo` (зв'язок «Один до багатьох»), визначаючи завершений маршрут для одного режиму пересування з повними даними `tsp_route`, `full_route` та

total_time_s.

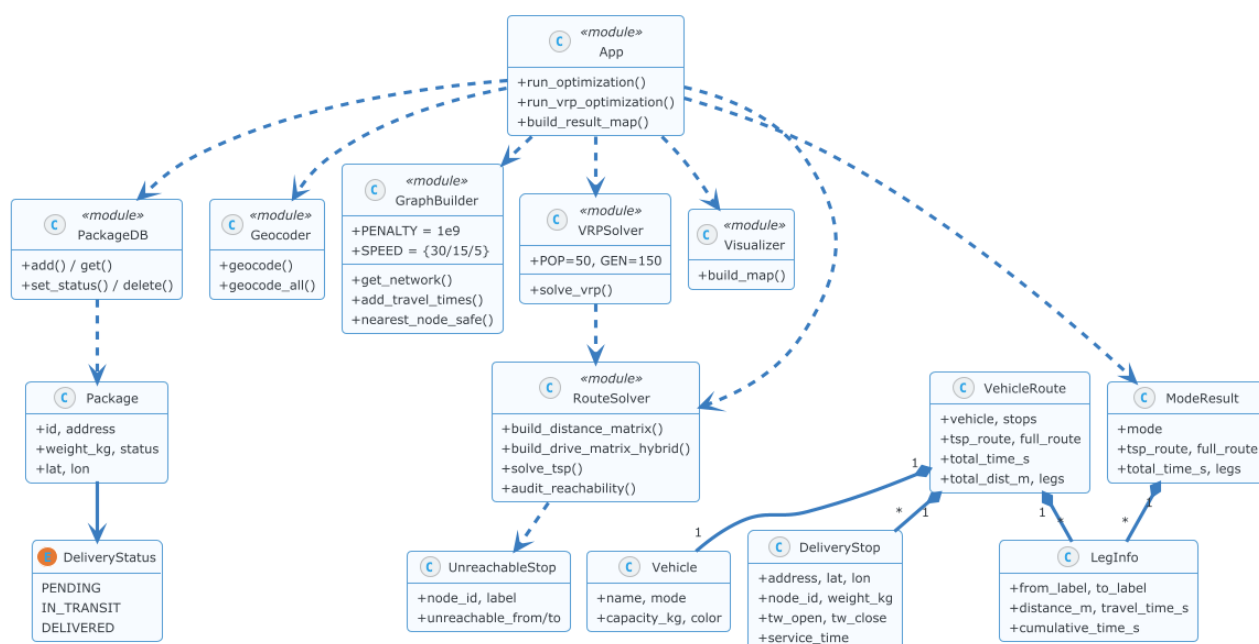


Рисунок 2.2 – Діаграма класів системи оптимізації вантажоперевезень

Пакет «Парк ТЗ» включає два класи. Vehicle описує транспортний засіб (атрибути name, mode, capacity_kg, color), а VehicleRoute пов'язує конкретний транспортний засіб зі своїм впорядкованим списком зупинок. Саме VehicleRoute є вихідною сутністю при розв'язанні багатотранспортної задачі CVRPTW.

Пакет «Транспортна мережа» реалізує інфраструктурний шар. Клас TransportGraph зі стереотипом «MultiDiGraph» агрегує множини NodeData (вузли з атрибутами id, y, x – широта і довгота) та EdgeData (ребра з атрибутами length, highway, travel_time загальний та три модальні travel_time_drive/bike/walk, а також модальні теги motor_vehicle, bicycle, foot, oneway). Така структура з трьома незалежними модальними часами на одному ребрі дозволяє підтримувати мультимодальну маршрутизацію без дублювання геометрії графа.

Пакет «База пакунків» інкапсулює персистентний шар. Клас PackageDB приховує деталі реалізації SQLite (поле _conn має модифікатор приватності) і пропонує публічний інтерфейс CRUD-операцій (add_package(), set_status(), get_by_status(), get_all(), delete()). Клас Package (атрибути id, address, weight_kg,

status, created_at, lat, lon) пов'язаний з перерахуванням DeliveryStatus, що приймає одне з трьох значень: PENDING, IN_TRANSIT, DELIVERED.

Пакет «Геокодування» містить допоміжний клас Location (address, lat, lon, node_id), який є проміжним результатом виклику геокодувального сервісу Nominatim. Між Location та DeliveryStop встановлено відношення залежності зі стереотипом «створюється з», що відображає сценарій перетворення геокодованої локації у точку доставки.

Окремо від пакетів зображено клас UnreachableStop (атрибути node_id, label, unreachable_from, unreachable_to, is_isolated та метод summary()), що відповідає за діагностику та звітування про точки доставки, які не вдалося досягти жодним з режимів пересування. Цей клас використовується модулем аудиту досяжності та інтегрується з підсистемою попереджень користувача.

Між класами використано три типи структурних відношень: композиція для випадків, коли життєвий цикл частини повністю залежить від цілого; агрегація для слабкого володіння, де частини можуть існувати незалежно; та залежність для випадків тимчасового використання одного класу іншим без структурної прив'язки.

Побудована модель є основою для подальшого опису структури бази даних у п. 2.2 та узгоджується з модульною архітектурою системи, представленою у п. 2.1.3.

2.1.3 Структурна модель – модульна архітектура

Структурну модель системи DeliveryIQ доповнено діаграмою класів модульної структури, побудованою у нотації UML 2.5. На відміну від доменної моделі (п. 2.1.2), яка описує сутності предметної області, модульна діаграма відображає декомпозицію вихідного коду на функціональні модулі та формалізує службові залежності між ними. Кожен пакет на діаграмі відповідає

одному файлу-модулю Python, а класи всередині пакета зображено зі стереотипом «module» та переліком публічних констант, атрибутів і функцій. Діаграму наведено на рисунку 2.3.

На діаграмі виділено шість модулів, що утворюють шарувату архітектуру із чітко визначеними обов'язками.

Модуль `app` (стереотип «Streamlit UI») є центральним контролером системи та точкою входу веб-застосунку. Він зберігає глобальні константи `DEFAULT_CITY = "Dnipro, Ukraine"`, `LAST_METER_THRESHOLD_M = 100.0` та посилення на синглтон-екземпляри геокодувального клієнта `_geolocator` і бази пакунків `_pkg_db`. Модуль експонує функції кешованої побудови мережі (`cached_network()`, `cached_mode_graphs()`), фасадні функції геокодування (`forward_geocode()`, `reverse_geocode()`), а також дві ключові функції-оркестратори: `run_optimization()` для одиничного транспортного засобу та `run_vrp_optimization()` для багатотранспортної задачі. Допоміжні функції `build_selection_map()` і `build_result_map()` формують Folium-карти для відображення на сторінці.

Модуль `vrp_solver` реалізує ядро генетичного алгоритму для CVRPTW. Окрім константи `VEHICLE_COLORS` (палітра кольорів маркерів автопарку), модуль експонує публічну функцію `solve_vrp()`, що приймає список зупинок, депо, флот та модальні графи й повертає сукупність маршрутів `VehicleRoute` разом зі списком попереджень. Внутрішні службові функції – `_assign_stops_to_modes()` (Фаза 1, призначення зупинок до сумісних режимів), `_build_ga_routes()` (Фаза 2, генетичний алгоритм) та `_decode_chromosome()` (Split-декодер хромосоми) – приховано через префікс підкреслення.

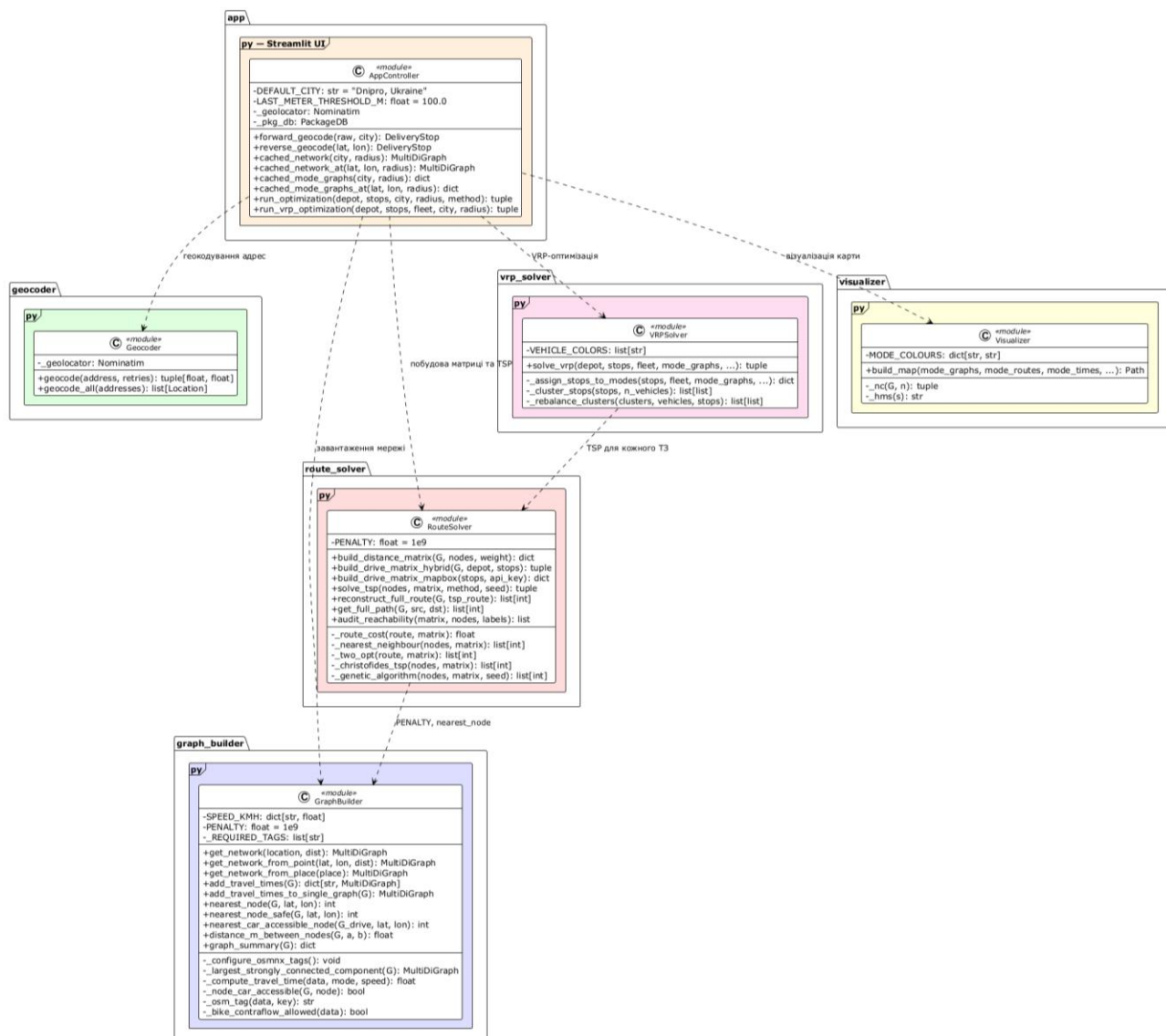


Рисунок 2.3 – Діаграма класів модульної архітектури системи

Модуль `route_solver` інкапсулює побудову матриць відстаней та алгоритми розв'язання задачі комівояжера. Експортує константу `PENALTY = 1e9`, для маркування недосяжних пар вузлів, функції побудови матриць трьох типів (`build_distance_matrix()` – статична через Дейкстру, `build_drive_matrix_hybrid()` – гібридна з пішим «останнім метром», `build_drive_matrix_mapbox()` – динамічна через Mapbox API), уніфіковану функцію `solve_tsp(method=...)` з підтримкою чотирьох алгоритмів (Nearest Neighbour, Christofides, 2-opt, Genetic), реалізованих у приватних функціях `_christofides()`, `_two_opt()`, `_genetic()`. Допоміжні `reconstruct_full_route()` та `get_full_path()` відновлюють повну послідовність вузлів на дорожньому графі, а `audit_reachability()` виявляє

ізолювані пари.

Модуль `graph_builder` відповідає за роботу з картографічними даними. Зберігає словник модальних швидкостей `SPEED_KMH = {drive: 30, bike: 15, walk: 5}` (км/год), константу `PENALTY` та перелік `REQUIRED_TAGS` для конфігурації `OSMnx`. Публічні функції покривають повний цикл роботи з графом: завантаження мережі (`get_network()`, `get_network_from_point()`), проставлення модальних часів (`add_travel_times()` – повертає три копії графа, `add_travel_times_to_single_graph()` – єдиний граф з атрибутами `travel_time_drive/bike/walk`), прив'язку координат до найближчого вузла (`nearest_node()`, `nearest_node_safe()`, `nearest_car_accessible_node()`), вимірювання відстані між вузлами (`distance_m_between_nodes()`) та діагностику (`graph_summary()`). Приватні функції `_configure_osmnx_tags()`, `_prune_to_lscc()`, `_node_car_accessible()` забезпечують внутрішню логіку фільтрації та обрізки графа.

Модуль `geocoder` є тонкою обгорткою над бібліотекою `GeoPy` і експортує спрощений інтерфейс з двох функцій – `forward_geocode(address, city)` та `reverse_geocode(lat, lon)` – що повертають кортежі координат і текстові адреси. Виділення цього модуля окремо дозволяє замінити провайдера геокодування (наприклад, на `Google Maps Geocoding API`) без зміни решти коду.

Модуль `visualizer` агрегує засоби побудови інтерактивних карт. Зберігає словник модальних кольорів `MODE_COLOURS`, експортує функцію `build_map()` для формування `Folium`-карти з шарами `AntPath`, а також приватні допоміжні функції `_ant()` (формування стилю анімованої лінії) та `_hms()` (форматування часових тривалостей).

Між модулями на діаграмі встановлено п'ять відношень залежності зі стереотипом «use»: `app` залежить від усіх п'яти службових модулів (`vrp_solver`, `route_solver`, `graph_builder`, `geocoder`, `visualizer`); `vrp_solver` використовує `route_solver` (для розв'язання TSP кожного транспортного засобу окремо) та `graph_builder` (для отримання `nearest_car_accessible_node` у режимі гібридної доставки); `route_solver` залежить від `graph_builder` (узгодження константи

PENALTY та використання `nearest_node`). Така структура утворює орієнтований ациклічний граф залежностей, що відповідає принципу шаруватої архітектури та запобігає виникненню циклічних імпортів.

2.1.4 Поведінкова модель – геокодування адреси

Для опису динамічного аспекту функціонування системи в межах одного бізнес-сценарію застосовано діаграми послідовностей (sequence diagrams) UML. Першу таку діаграму присвячено сценарію геокодування адреси доставки – це найчастіше виконуваний сценарій у роботі оператора та критична точка інтеграції системи із зовнішнім сервісом Nominatim. Діаграму послідовності наведено на рисунку 2.4.

На діаграмі виділено три лінії життя (lifeline): актора Оператор, програмний модуль `app.py` (стереотип «Streamlit UI») та зовнішній актор Nominatim API (з реалізаційним коментарем «geopy», що позначає клієнтську бібліотеку доступу). Сценарій складається з двох незалежних потоків: основного (введення адреси вручну) та альтернативного (додавання точки кліком на карту), розділених на діаграмі горизонтальною межею.

Основний сценарій – текстове введення адреси. Послідовність розпочинається з повідомлення «Вводить адресу в поле "Add delivery address"», яке оператор надсилає до модуля `app.py`. Модуль виконує `self-message` – внутрішній виклик функції `_lock(address, city)`, що дописує до введеного рядка локацію міста (наприклад, додає суфікс «Dnipro, Ukraine»), забезпечуючи прив'язку результату геокодування до конкретного міста та усуваючи неоднозначність омонімічних топонімів. Далі модуль викликає синхронний метод `geocode(query, timeout=10)` сервісу Nominatim і чекає на відповідь.

Після отримання відповіді управління переходить до блоку альтернатив, який моделює три можливі сценарії розгалуження:

– Гілка «Адресу знайдено» – найімовірніший сценарій. Nominatim повертає об'єкт `result(address, lat, lon)`. Модуль `app.py` створює екземпляр доменного класу `DeliveryStop` з атрибутами `address=result.address`, `lat=result.latitude`, `lon=result.longitude` та `source="typed"`. Створений об'єкт додається до колекції точок доставки у сесійному стані `Streamlit` (`st.session_state["stops"].append(stop)`). Завершальним повідомленням до оператора є оновлення інтерфейсу: «Маркер на карті + запис у списку доставок».

– Гілка «Адресу не знайдено» спрацьовує, коли Nominatim повертає `None`. Модуль формує повідомлення про помилку «Could not geocode address» і виводить його у вигляді червоного банера. Дані у сесійний стан не записуються – інваріант «список точок містить лише валідні геокодовані адреси» зберігається.

– Гілка «Таймаут» активується при перевищенні часу очікування (10 секунд) – клієнтська бібліотека викидає виняток `GeocoderTimedOut`, який перехоплюється модулем і трансформується у повідомлення «Geocoder timed out» для оператора. Така обробка забезпечує стійкість системи до тимчасових проблем з мережевим з'єднанням та перевантаження сервера Nominatim.

Альтернативний сценарій – додавання точки кліком на карту. Оператор клікає на довільну точку інтерактивної карти, і браузер передає координати (`lat`, `lon`) модулю `app.py` через механізм `Streamlit`-сесії. Модуль виконує зворотне геокодування – викликає метод `reverse((lat, lon), language="en")` сервісу Nominatim, який за координатами повертає найближчу адресу у вигляді текстового рядка. На основі отриманого результату створюється екземпляр `DeliveryStop` з атрибутами `address=result.address`, `lat=lat`, `lon=lon` та `source="map_click"` (атрибут `source` дозволяє згодом розрізняти точки, додані текстово і кліком – наприклад, для аналітичних звітів). Об'єкт додається до сесійного стану, а на карті з'являється новий маркер з відображенням зворотно-геокодованої адреси.

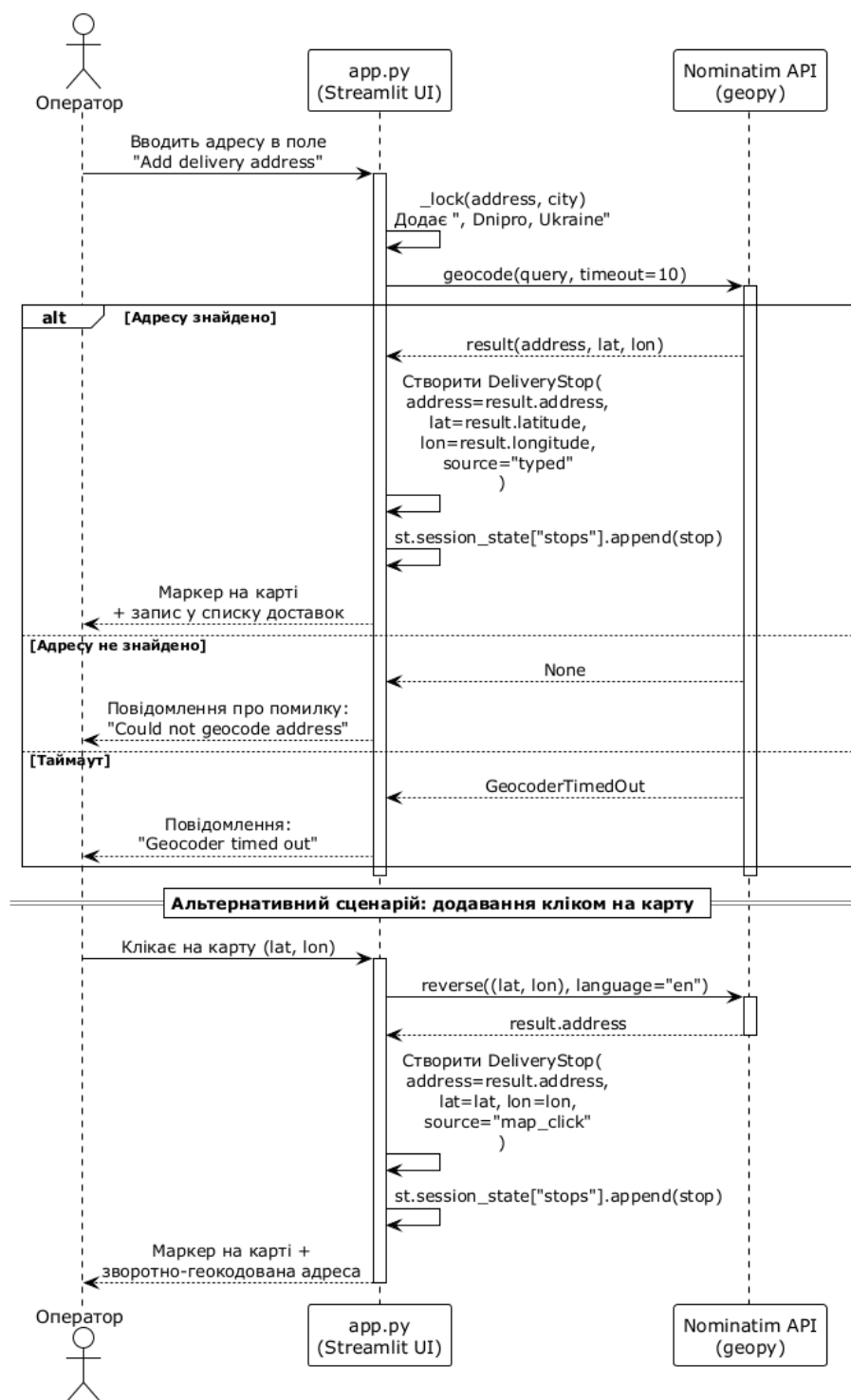


Рисунок 2.4 – Діаграма послідовності геокодування адреси

Доступ до Nominatim ізольовано у модулі `geocoder.py`. Решта системи бачить лише дві функції – `forward_geocode()` і `reverse_geocode()` – і нічого не знає ні про HTTP-запити, ні про таймаути, ні про специфіку відповідей сервісу.

2.1.5 Поведінкова модель – оптимізація одного транспортного засобу

Для документування динаміки виконання основного розрахункового сценарію – побудови оптимального маршруту для одного транспортного засобу із порівнянням трьох режимів пересування – побудовано розгорнуту діаграму послідовностей. Цей сценарій є найбільш обчислювально складною операцією системи і об'єднує взаємодію всіх п'яти службових модулів між собою та з зовнішнім сервісом OpenStreetMap. Діаграму послідовності наведено на рисунку 2.5.

На діаграмі представлено шість ліній життя: актор Оператор, чотири внутрішні модулі – `app.py` (Streamlit UI), `graph_builder.py` (GraphBuilder), `route_solver.py` (RouteSolver), `visualizer.py` (Visualizer) – та зовнішній актор OSM (Overpass API) як джерело первинних картографічних даних. Для покращення читабельності повний цикл оптимізації розділено на шість фаз, кожна з яких винесена в окрему секцію зі стереотипом «ref».

Фаза 1 – Завантаження мережі. Сценарій ініціюється повідомленням «Натискає Optimize Routes» від оператора до `app.py`. Контролер викликає функцію `cached_network_at(depot.lat, depot.lon, radius)` модуля `graph_builder.py`. Перед формуванням HTTP-запиту GraphBuilder виконує внутрішній виклик `_configure_osmnx_tags()`, що налаштовує бібліотеку OSMnx на збереження ключових модальних тегів (`bicycle`, `motor_vehicle`, `foot`, `cycleway`, `oneway`). Далі модуль робить синхронний запит `ox.graph_from_point(lat, lon, dist, "all")` до Overpass API, отримує сирий мультимодальний граф `G_raw` типу `MultiDiGraph` та виконує self-message `_largest_strongly_connected_component(G_raw)`, що відсікає ізольовані компоненти графа й залишає лише найбільшу сильно зв'язну компоненту (LSCC). Очищений граф `G` повертається до контролера.

Фаза 2 – Прив'язка адрес до вузлів. Контролер у комбінованому фрагменті зі стереотипом «loop» для кожної адреси (депо та усі точки доставки) викликає функцію `nearest_node(G, lat, lon)`, що повертає `node_id` – числовий ідентифікатор

найближчого вузла графа. Після виходу з циклу `app.py` виконує `self-message` Дедуплікація вузлів (`dict.fromkeys`) – це усуває потенційні колізії, коли дві різні адреси внаслідок невисокої точності геокодування прив'язалися до одного й того ж вузла OSM-графу.

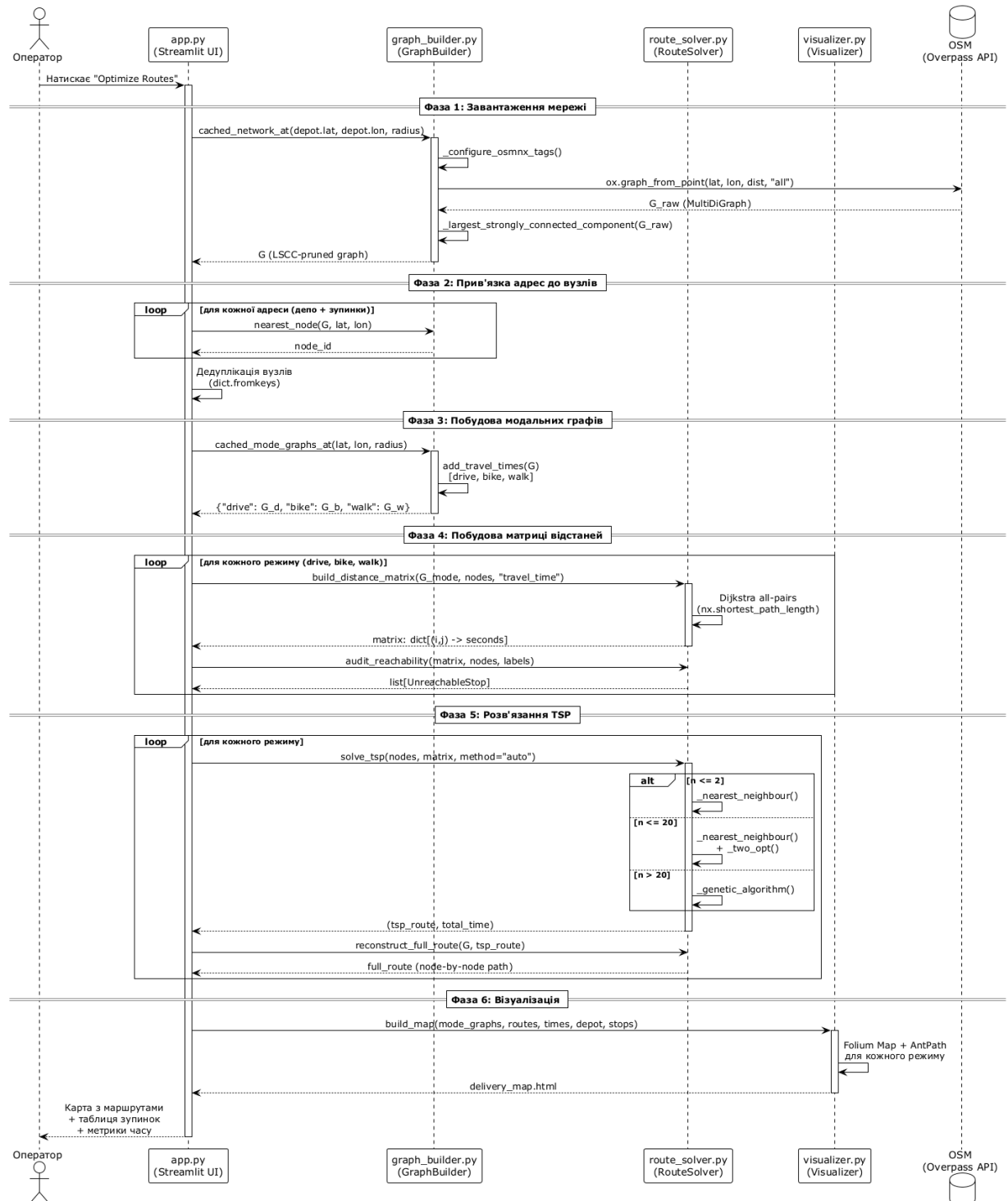


Рисунок 2.5 – Діаграма послідовності процесу оптимізації маршруту

Фаза 3 – Побудова модальних графів. Контролер викликає `cached_mode_graphs_at(lat, lon, radius)` модуля `GraphBuilder`, який внутрішньо виконує `add_travel_times(G)` – створення трьох глибоких копій графа із проставленням атрибута `travel_time` на ребрах за модальними швидкостями (`drive` – 30 км/год, `bike` – 15, `walk` – 5). Повертається словник `{"drive": G_d, "bike": G_b, "walk": G_w}`. Кешування цієї операції критичне, оскільки обхід графа з кількома десятками тисяч ребер для проставлення модальних часів займає 2-5 секунд.

Фаза 4 – Побудова матриці відстаней. У циклі `loop` для кожного з трьох режимів (`drive`, `bike`, `walk`) контролер викликає `build_distance_matrix(G_mode, nodes, "travel_time")` модуля `RouteSolver`. Той виконує self-message «Dijkstra all-pairs» (`nx.shortest_path_length`) – обчислення матриці найкоротших часів між усіма парами вузлів. Результат повертається у вигляді словника `dict[(i, j) → seconds]`. Після побудови матриці контролер викликає допоміжну `audit_reachability(matrix, nodes, labels)`, що повертає список об'єктів `UnreachableStop` – точок, недосяжних у даному режимі (наприклад, велосипедна доставка через пішохідну зону).

Фаза 5 – Розв'язання задачі TSP. Для кожного режиму контролер викликає уніфіковану функцію `solve_tsp(nodes, matrix, method="auto")`. У режимі автоматичного вибору алгоритму `RouteSolver` застосовує комбінований фрагмент «alt» із трьома гілками: для 2 та менше точок доставки – простий `_nearest_neighbour()`; від 2 до 20 – комбінація `_nearest_neighbour()` та `_two_opt()`; для 20 та більше – `_genetic_algorithm()`. Така адаптивна стратегія мінімізує загальний час обчислень: для малих задач відсутній накладний час популяційної метаевристики, для великих – гарантується якість розв'язку. Результат (`tsp_route`, `total_time`) повертається до контролера, який далі викликає `reconstruct_full_route(G, tsp_route)` для відновлення повного `full_route` – послідовності всіх проміжних вузлів графа, придатної для рендерингу.

Фаза 6 – Візуалізація. Контролер передає всі підготовлені дані (`mode_graphs`, `routes`, `times`, `depot`, `stops`) до `build_map()` модуля `Visualizer`. Той

формує об'єкт Folium Map, накладає на нього три анімовані шари AntPath (по одному для кожного режиму, з різними кольорами), маркери депо та точок доставки, після чого серіалізує карту у HTML-документ `delivery_map.html`, який повертається контролеру. Завершальним повідомленням оператор отримує «Карта з маршрутами, таблиця зупинок, метрики часу».

2.1.6 Поведінкова модель – оптимізація багатьох ТЗ

Сценарій багатотранспортної оптимізації є найскладнішим з точки зору алгоритмічної логіки, оскільки до задачі побудови маршруту додаються етапи розподілу зупинок між транспортними засобами з урахуванням їх режимів та вантажомісткості. Діаграма послідовностей цього сценарію демонструє трифазну архітектуру VRP-розв'язувача та його взаємодію з іншими модулями системи. Діаграму наведено на рисунку 2.6.

На діаграмі представлено п'ять ліній життя: актор Оператор, контролер `app.py` (Streamlit UI), ключовий розв'язувач `vrp_solver.py` (VRPSolver), модуль `route_solver.py` (RouteSolver) та допоміжний `graph_builder.py` (для виклику `nearest_car_accessible_node` у режимі гібридної доставки). Сценарій ініціюється повідомленням «Налаштовує парк ТЗ (тип, місткість) та натискає Optimize Fleet» від оператора. Контролер виконує єдиний фасадний виклик `solve_vrp(depot, stops, fleet, mode_graphs, G_raw, radius)` модуля VRPSolver, що повертає кортеж `(list[VehicleRoute], warnings)`. Внутрішня структура цього виклику розділена на три послідовні фази, винесені в окремі секції стереотипу «ref».

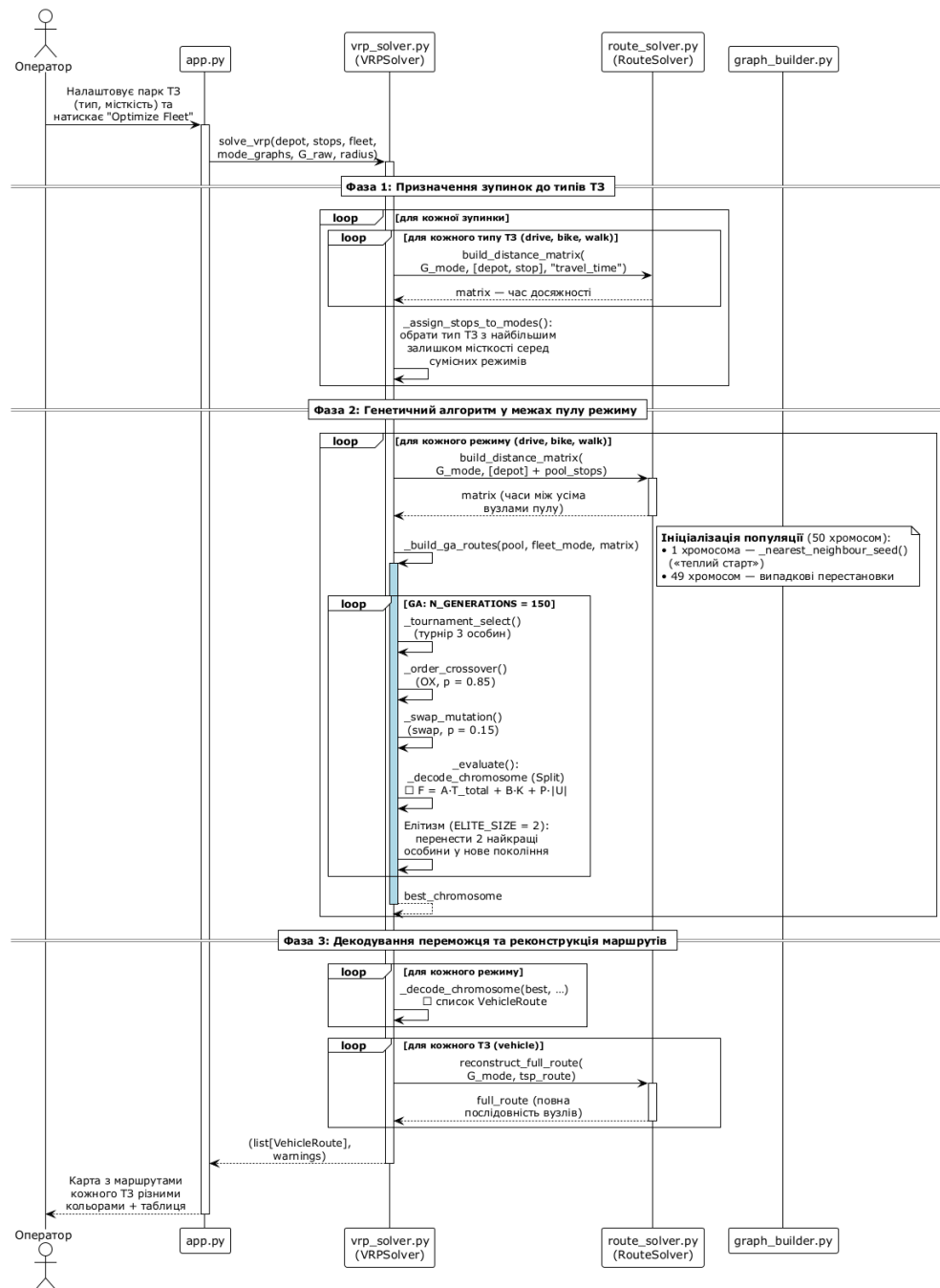


Рисунок 2.6 – Діаграма послідовності процесу оптимізації декількох ТЗ

Фаза 1 – Призначення зупинок до типів ТЗ. Перший етап вирішує задачу багатокритеріального розподілу: для кожної точки доставки потрібно обрати такий тип транспортного засобу, який фізично здатен її досягти (модальна сумісність ребер дорожньої мережі), та має достатній залишок вантажомісткості. На діаграмі цей етап представлено вкладеним подвійним циклом «loop». Зовнішній цикл «для кожної зупинки» ітерується по всіх точках доставки. Внутрішній цикл «для кожного типу ТЗ (drive, bike, walk)» викликає

`build_distance_matrix(G_mode, [depot, stop], "travel_time")` модуля `RouteSolver`, що повертає мінімальну `matrix` – час досяжності зупинки з депо для даного режиму. Якщо матриця містить значення `PENALTY = 1e9` секунд, зупинка вважається недосяжною для цього режиму і виключається з кандидатів. Після завершення внутрішнього циклу `VRPSolver` виконує `self-message` «Обрати тип ТЗ з найбільшим залишком місткості серед сумісних режимів», що реалізує жадібну стратегію балансування завантаження автопарку.

Фаза 2 – Генетичний алгоритм у межах пулу режиму (ядро розв'язувача). Після призначення кожній зупинці типу транспортного засобу виникає основна оптимізаційна підзадача – впорядкувати зупинки кожного пулу між конкретними транспортними засобами того ж режиму так, щоб мінімізувати сумарний час руху за дотримання обмежень вантажомісткості та часових вікон. Саме на цьому етапі задіюється головний обчислювальний механізм системи – генетичний алгоритм (GA), детально описаний у п. 2.3. У циклі «для кожного режиму (`drive`, `bike`, `walk`)» `VRPSolver` передає пул зупинок цього режиму у внутрішню функцію `_build_ga_routes()`, яка інкапсулює повний цикл еволюційного пошуку.

Перед першою генерацією формується початкова популяція з 50 хромосом – кодованих як перестановки індексів зупинок («`giant tour`») без явних маркерів меж маршрутів. Одна хромосома створюється «теплим стартом» через жадібну евристику найближчого сусіда (`_nearest_neighbour_seed()`), решта – випадковими перестановками, що забезпечує початкове генетичне різноманіття. На кожній ітерації виконуються послідовні `self-message`: `_tournament_select()` (турнірна селекція трьох особин), `_order_crossover()` (Order Crossover, ОХ, з імовірністю 0.85), `_swap_mutation()` (обмін двох випадкових генів, імовірність 0.15) та `_evaluate()`. Останній викликає ключовий Split-декодер `_decode_chromosome()`, що проходить перестановку зліва направо і жадібно нарізає її на сегменти, кожен з яких становить допустимий маршрут одного транспортного засобу – як тільки додавання чергової зупинки порушує обмеження місткості (2.10) або часового вікна, поточний маршрут «закривається» поверненням до депо, а зупинка переходить у наступний сегмент. Така структура декодера гарантує, що будь-яка

хромосома-перестановка автоматично відображається у допустимий план маршрутів, а отже оператори OX та swar-мутації ніколи не продукують некоректних нащадків. Двох найкращих особин кожного покоління переноситься у наступне без змін (елітизм, `ELITE_SIZE = 2`) – це гарантує монотонне неухудшення найкращого знайденого розв'язку.

Фаза 3 – Декодування переможця та реконструкція повних маршрутів. Після завершення `N_GENERATIONS = 150` ітерацій або стагнації найкращої придатності протягом останніх поколінь алгоритм повертає хромосому-переможця. У циклі «для кожного режиму» `VRPSolver` виконує фінальне декодування цієї хромосоми у сукупність об'єктів `VehicleRoute` (по одному на кожен задіяний транспортний засіб), після чого послідовно викликає `reconstruct_full_route(G_mode, tsp_route)` модуля `RouteSolver` для відновлення повної послідовності проміжних вузлів дорожнього графа, необхідної для рендерингу інтерактивної карти. Маршрут-рівневого виклику `solve_tsp()` на цій фазі не потрібно – порядок зупинок у межах кожного маршруту вже визначений безпосередньо хромосомою-переможцем.

Після завершення третьої фази `VRPSolver` формує сукупність об'єктів `VehicleRoute` (по одному на кожен задіяний транспортний засіб) та список діагностичних попереджень `warnings` (наприклад, «зупинка X не вмістилася у жоден ТЗ через обмеження місткості» або «зупинка Y присвоєна ТЗ Z з низькою досяжністю»). Сформований кортеж повертається контролеру `app.py`, а той у завершальному повідомленні до оператора передає «Карта з маршрутами кожного ТЗ різними кольорами + таблиця».

Описаний сценарій реалізує GA-центричну стратегію розв'язання `CVRPTW`: вся комбінаторна складність задачі (одночасний розподіл зупинок між транспортними засобами та впорядкування їх у межах кожного маршруту) інкапсульована у єдиному еволюційному циклі з Split-декодером. Така архітектура має дві принципові переваги порівняно з класичними двофазними евристиками «cluster-first, route-second» (наприклад, кластеризація K-Means з подальшим розв'язанням TSP у кожному кластері): GA одночасно оптимізує і

розподіл, і впорядкування, тоді як двофазні підходи фіксують розподіл після першої фази і не можуть його переглянути; Split-декодер вбудовує обмеження місткості й часових вікон безпосередньо у функцію придатності, що усуває потребу в окремих кроках постобробки. Жадібна евристика найближчого сусіда збережена в системі – але не як самостійний алгоритм, а як генератор однієї хромосоми «теплого старту», що дозволяє ГА починати пошук з уже непоганого розв'язку і пришвидшує збіжність на 15-20 % порівняно з повністю випадковою ініціалізацією. Таким чином, у програмному продукті реалізовано єдиний основний алгоритм оптимізації – генетичний, тоді як класичні конструктивні евристики (Nearest Neighbour, 2-opt, Кларка-Райта) задіяні лише як базові орієнтири для порівняльних чисельних експериментів у п. 3.7.

Узагальнено динамічну поведінку обох сценаріїв (одно- та багатотранспортного) у єдиному наскрізному процесі описує діаграма діяльності, яка розглядається у наступному підпункті.

2.1.7 Динамічна модель – діаграма діяльності

Для узагальненого опису повного процесу оптимізації як послідовності логічно пов'язаних дій, із явним представленням точок прийняття рішень, паралельних потоків обробки та сценаріїв обробки помилок, побудовано діаграму діяльності (activity diagram) у нотації UML. На відміну від діаграм послідовностей (п. 2.1.4-2.1.6), що показують міжмодульну взаємодію в часі, діаграма діяльності абстрагується від конкретних ліній життя і фокусується на потоці управління та обробці даних на рівні бізнес-процесу. Діаграму наведено на рисунку 2.7.

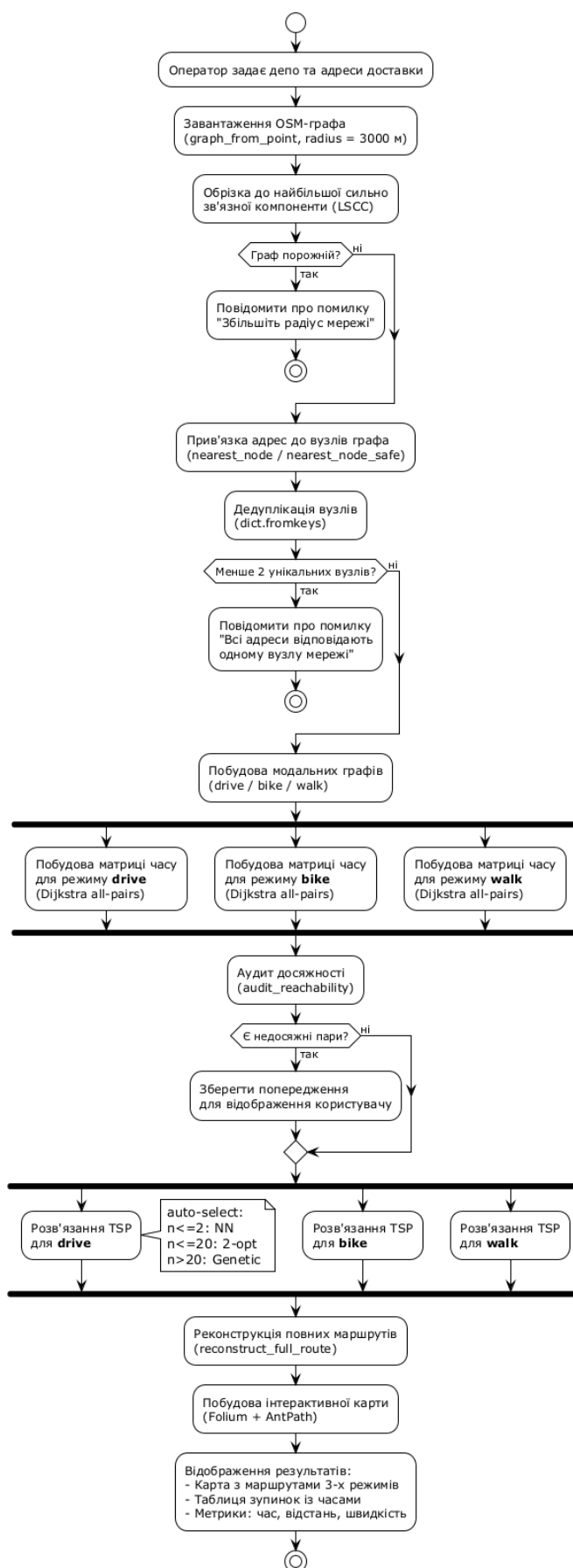


Рисунок 2.7 – Діаграма діяльності процесу оптимізації маршруту доставки

Діаграма починається з початкового вузла (заповнений чорний кружок) і завершується кінцевим вузлом (білий кружок з чорною облямівкою). Між ними розгорнуто послідовність із п'ятнадцяти прямокутних вузлів-дій, з'єднаних потоками управління. Процес можна логічно поділити на чотири стадії: підготовка даних, побудова обчислювальної інфраструктури, паралельні обчислення для трьох режимів, синтез та подання результатів.

Стадія I – Підготовка даних. Процес ініціюється дією «Оператор задає депо та адреси доставки» – це єдина дія, що вимагає інтерактивного вводу.

Після її завершення керування передається системній дії «Завантаження OSM-графа (`graph_from_point, radius = 3000 м`)», що формує HTTP-запит до Overpass API та отримує сирий мультимодальний граф. Наступна дія «Обрізка до найбільшої сильно зв'язної компоненти (LSCC)» виконує постобробку: видаляє відірвані частини графа, які могли потрапити в межі радіусу через фрагментацію OSM-даних.

Перша точка прийняття рішення (ромб) – «Граф порожній?» – перевіряє результат обрізки. Гілка «так» веде до дії «Повідомити про помилку: Збільшіть радіус мережі» і завершує процес у кінцевому вузлі. Гілка «ні» продовжує виконання. Така перевірка є критичною для коректної роботи системи в умовах, коли депо розташоване в географічній зоні з низькою щільністю OSM-даних (наприклад, у промисловій зоні на околиці міста).

Друга стадія містить дії «Прив'язка адрес до вузлів графа (`nearest_node / nearest_node_safe`)» – для кожної введеної адреси визначається найближчий вузол LSCC-обрізаного графа – та «Дедуплікація вузлів (`dict.fromkeys`)», що усуває колізії однакових прив'язок. Друга точка прийняття рішення «Менше 2 унікальних вузлів?» захищає від випадку, коли всі точки доставки виявилися занадто близько одна до одної (або до депо) і відобразилися в один вузол мережі. У такому випадку задача втрачає сенс – алгоритм формує повідомлення «Всі адреси відповідають одному вузлу мережі» і коректно завершує процес.

Стадія II – Побудова обчислювальної інфраструктури. Дія «Побудова модальних графів (`drive / bike / walk`)» виконує триразове клонування LSCC-

графа із проставленням модальних часів на ребрах згідно правил, описаних у п. 2.3 та таблиці модальної доступності.

Далі діаграма містить перший fork-вузол (товста горизонтальна лінія), що розгалужує єдиний потік управління на три паралельні потоки. Кожен з них виконує дію «Побудова матриці часу для режиму X (Dijkstra all-pairs)», де X – drive, bike або walk. Паралелізація на цьому етапі є природним кроком, оскільки три модальні матриці є незалежними обчисленнями і можуть виконуватися конкурентно на багатоядерних процесорах.

Після завершення всіх трьох потоків керування синхронізується у join-вузлі (товста горизонтальна лінія) і переходить до дії «Аудит досяжності (audit_reachability)», що сканує матриці на наявність значень PENALTY. Третя точка прийняття рішення «Є недосяжні пари?» – без виходу з процесу: гілка «так» додає системну дію «Зберегти попередження для відображення користувачу», після чого обидві гілки об'єднуються.

Стадія III – Паралельне розв'язання задач TSP. Аналогічно стадії побудови матриць, цей етап використовує конструкцію fork-join. Три паралельні дії «Розв'язання TSP для drive / bike / walk» виконуються одночасно. На діаграмі поряд з гілкою drive розміщено блок-коментар з описом адаптивної логіки auto-select: для 2 або менше точок доставки – Nearest Neighbour; від 2 до 20 – 2-opt; більше ніж 20 – Genetic Algorithm.

Стадія IV – Синтез і подання результатів. Після join-вузла процес послідовно проходить три дії: «Реконструкція повних маршрутів (reconstruct_full_route)» – перетворення TSP-маршрутів зі скороченим набором вузлів на повну послідовність проміжних вузлів графа, придатну для рендерингу; «Побудова інтерактивної карти (Folium + AntPath)» – формування HTML-карти з анімованими шарами; «Відображення результатів» з блоком-коментарем, що деталізує склад вихідних артефактів – карта з маршрутами трьох режимів, таблиця зупинок з часами та метрики (час, відстань, швидкість). Процес завершується у кінцевому вузлі.

2.2 Інформаційне забезпечення

Інформаційне забезпечення системи охоплює сукупність даних, необхідних для функціонування алгоритмів маршрутизації, та засоби їхньої організації, зберігання й оновлення. Інформаційну модель розроблюваної підсистеми доцільно розглянути на трьох рівнях абстракції: концептуальному, логічному і фізичному.

2.2.1 Концептуальний рівень – сутності предметної області

Інформаційна модель системи концентрується навколо центральної сутності предметної області – пакунка (Package), що представляє фізичну одиницю вантажу, який підлягає доставці. На відміну від доменної моделі (п. 2.1.2), де описано статичну структуру всіх класів системи, на концептуальному рівні деталізується семантика пакунка як об'єкта зі змінним у часі станом та формалізується його поведінка засобами діаграми станів.

Кожен екземпляр пакунка містить сім атрибутів, що утворюють повний інформаційний образ замовлення:

- ідентифікатор – глобально унікальний код, що дозволяє посилатися на пакунок з будь-якого модуля системи без ризику колізій;
- адреса доставки – текстове представлення точки призначення у форматі, прийнятному для геокодувального сервісу Nominatim;
- вага вантажу – додатне дійсне число у кілограмах, що враховується в обмеженнях вантажомісткості транспортних засобів;
- стан життєвого циклу – поточна фаза обробки замовлення, що приймає одне з трьох значень: «очікує», «в дорозі», «доставлено»;
- момент створення – часова мітка реєстрації замовлення в системі,

потрібна для впорядкування пакунків та формування історичної статистики;

– географічні координати – пара широта/довгота, що заповнюються після успішного геокодування адреси і можуть бути відсутніми в момент створення пакунка.

Властивість змінного стану виокремлює Package серед інших сутностей системи. Якщо Vehicle, LegInfo або ModeResult є по суті незмінними (immutable) дата-об'єктами, що формуються одноразово і не модифікуються після створення, то пакунок проходить через декілька логічно різних станів протягом свого життєвого циклу – від моменту реєстрації замовлення до підтвердження доставки. Така поведінка природно описується кінцевим автоматом, формальне представлення якого наведено на рисунку 2.8. На діаграмі виділено три прості стани (зображені округленими прямокутниками з горизонтальною лінією-роздільником між іменем і внутрішніми діями), один початковий псевдостан та один кінцевий стан.

Стан «Очікує (PENDING)» є початковим станом життєвого циклу. У нього пакунок потрапляє відразу після створення в БД викликом методу add_package() модуля PackageDB. У внутрішніх інваріантах стану зафіксовано: «Пакунок створено в БД» та «Адреса задана, координати = None». Останнє є важливим уточненням: момент створення пакунка не обов'язково збігається з моментом успішного геокодування його адреси. Якщо сервіс Nominatim тимчасово недоступний або повертає помилку, пакунок все одно зберігається в базі для подальшого ручного дозаповнення координат оператором. Це забезпечує відмовостійкість системи у разі проблем зі сторонніми сервісами.

Стан «В дорозі (IN_TRANSIT)» характеризує пакунок, для якого оператор запустив оптимізацію маршруту і алгоритм успішно включив його у маршрут конкретного транспортного засобу. Внутрішні інваріанти стану: «Координати геокодовані», «Пакунок включено в маршрут ТЗ» та «Оптимізація виконана». Перехід з PENDING у IN_TRANSIT відбувається через подію set_status(IN_TRANSIT), спрацьовування якої обумовлене сторожовою умовою [Оператор запускає оптимізацію маршруту].

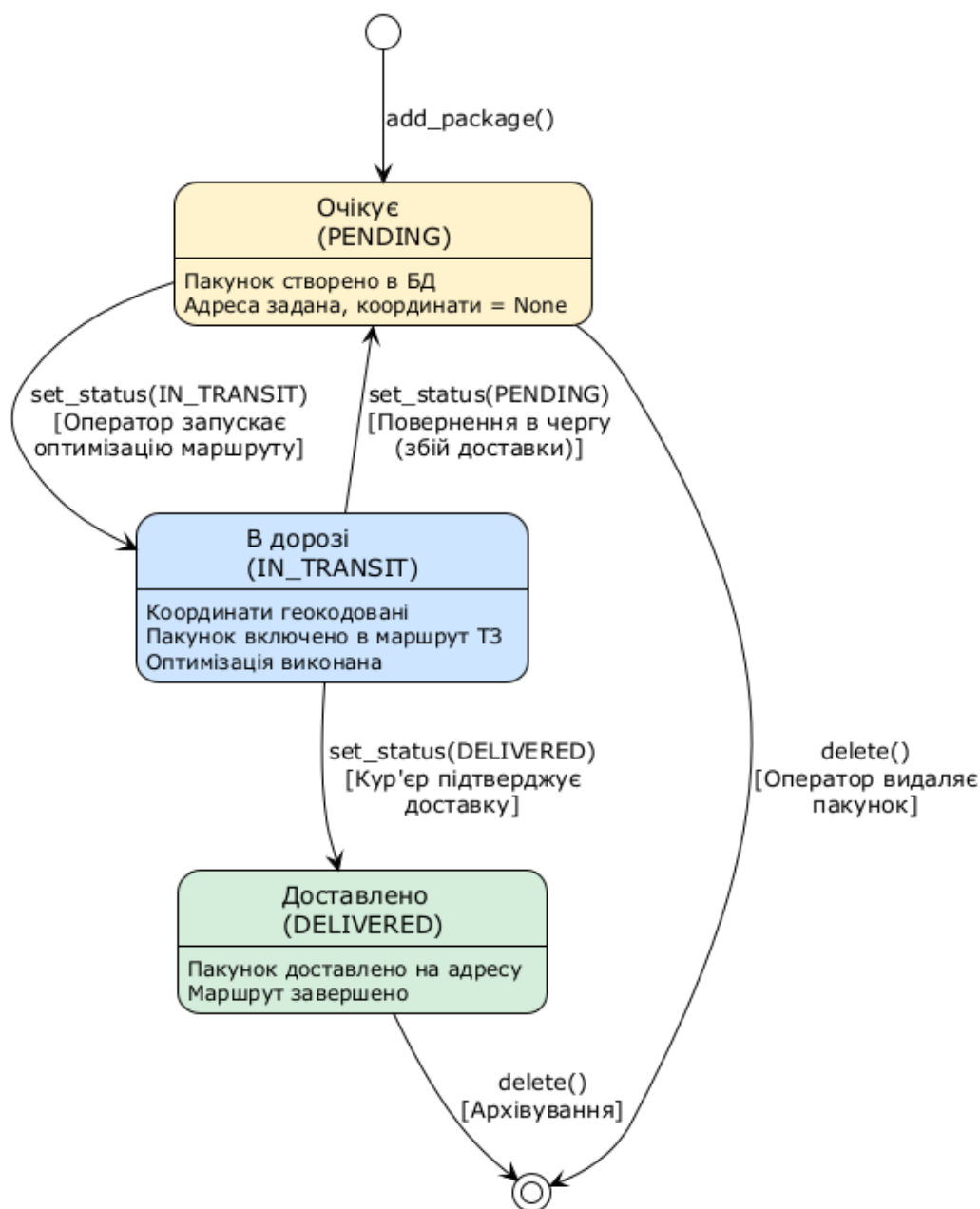


Рисунок 2.8 – Діаграма станів життєвого циклу пакунок

Стан «Доставлено (DELIVERED)» є термінальним семантичним станом – пакунок виконав свою функцію, кур'єр доставив його на цільову адресу та підтвердив це через інтерфейс системи. Внутрішні інваріанти: «Пакунок доставлено на адресу» та «Маршрут завершено». Перехід здійснюється подією `set_status(DELIVERED)` зі сторожовою умовою `[Кур'єр підтверджує доставку]`. Слід зауважити, що стан DELIVERED не є кінцевим у формально-UML сенсі: пакунок залишається в БД для подальшого аналізу та формування статистики, поки не буде явно видалений оператором.

На діаграмі представлено п'ять переходів між станами та два переходи в кінцевий вузол:

1. `PENDING` → `IN_TRANSIT` – основний прогресивний перехід, описаний вище;

2. `IN_TRANSIT` → `DELIVERED` – також прогресивний перехід, підтвердження доставки;

3. `IN_TRANSIT` → `PENDING` – регресивний перехід зі сторожовою умовою [Повернення в чергу (збій доставки)]. Він моделює реальну ситуацію, коли під час виконання маршруту виникає форс-мажорна обставина: клієнт відсутній за адресою, отримання неможливе через закритий доступ, у транспортного засобу виникла технічна несправність тощо. У таких випадках оператор повертає пакунок у стан очікування для повторного планування на наступний день. Наявність зворотного переходу робить кінцевий автомат неациклічним і дозволяє моделювати повторні спроби доставки без створення нового запису в БД;

4. `PENDING` → `final` – перехід зі сторожовою умовою [Оператор видаляє пакунок]. Спрацьовує при виклику методу `delete()` для пакунків, які так і не були відправлені (наприклад, помилкове замовлення або скасування клієнтом);

5. `DELIVERED` → `final` – перехід [Архівування]. Виконує фізичне видалення запису з активної таблиці; у виробничих умовах рекомендується замінити на «м'яке» видалення з перенесенням до архівної таблиці для збереження історії.

Описаний кінцевий автомат має три суттєві властивості, важливі для коректного функціонування системи. Він є детермінованим: з будь-якого стану і за будь-якої події існує не більше одного допустимого переходу. Це усуває неоднозначність обробки команд та спрощує тестування. Граф переходів є зв'язним: з будь-якого нетермінального стану можна досягти кінцевого вузла за скінченну кількість переходів, що гарантує відсутність «застряглих» пакунків у системі. Всі переходи захищені сторожовими умовами, які перевіряються на рівні бізнес-логіки в модулі `PackageDB`, що запобігає некоректним переходам

(наприклад, з PENDING безпосередньо у DELIVERED, оминаючи стан IN_TRANSIT) на рівні БД.

Програмно автомат реалізовано через метод `PackageDB.set_status(id, status)` з валідацією допустимих переходів та через перерахування `DeliveryStatus` (enum з трьома членами), що забезпечує статичну безпеку типів при роботі зі станами в коді.

2.2.2 Логічний рівень – реляційна схема та потоки даних

На логічному рівні концептуальна модель уточнюється у термінах реляційної алгебри з визначенням таблиць, ключів та зв'язків між сутностями. Через невелику кількість сутностей з персистентним станом (фактично лише `Package`) логічна схема системи `DeliveryIQ` є компактною і складається з однієї базової таблиці `packages` та одного перерахування `delivery_status`.

Таблиця `packages` має сім атрибутів, що безпосередньо відповідають концептуальним атрибутам сутності `Package`: первинний ключ `id`, обов'язкові атрибути `address` і `weight_kg`, дискримінант стану `status` з обмеженням за переліком `delivery_status`, часова мітка `created_at` та необов'язкова пара `(lat, lon)`. Структура таблиці у нотації ER-моделі (Entity-Relationship) наведена на рисунку 2.9. Усі стовпці є атомарними та функціонально залежними від первинного ключа, що задовольняє вимоги третьої нормальної форми (3НФ). Перерахування `delivery_status` формально могло б бути винесене в окрему таблицю-довідник зі зв'язком «один-до-багатьох», однак через незмінність переліку (три значення, заданих на рівні коду) такий рефакторинг визнано надлишковим – стан зберігається у стовпці типу `TEXT` з логічним обмеженням на боці бізнес-логіки.

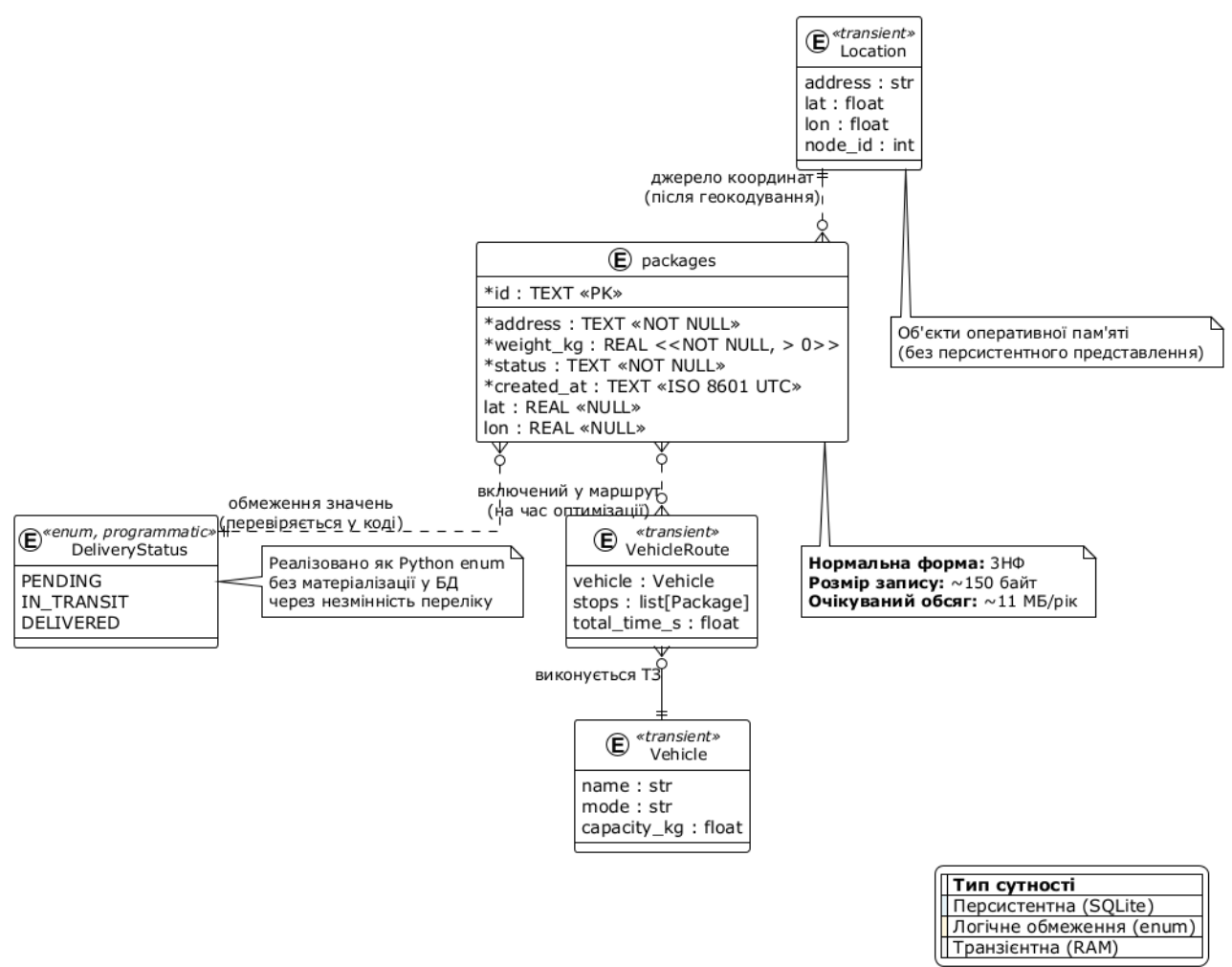


Рисунок 2.9 – ER-діаграма реляційної схеми системи

Між сутністю Package та іншими класами системи встановлено три типи логічних зв'язків, які не матеріалізуються у вигляді зовнішніх ключів через те, що відповідні класи (Vehicle, VehicleRoute, Location) є об'єктами оперативної пам'яті без персистентного представлення. Зв'язок Package↔Location є залежністю (dependency) – координати пакунка формуються з геокодованого Location, але після збереження зв'язок розривається. Зв'язок Package↔VehicleRoute є тимчасовою агрегацією – на час виконання маршруту VehicleRoute посилається на впорядкований список Package, але після завершення доставки цей зв'язок не зберігається. Зв'язок Package↔DeliveryStatus є реалізацією шаблону «довідник станів» з програмною верифікацією допустимих переходів у методі PackageDB.set_status().

Потоки даних. Інформаційна взаємодія сутності Package з іншими компонентами системи формалізується діаграмою потоків даних (DFD) рівня 1, наведеною на рисунку 2.10. Зовнішніми сутностями DFD є Оператор (джерело команд введення та змін стану) та зовнішній сервіс Nominatim (джерело геокодованих координат). Сховищем даних виступає таблиця packages. Між ними виділено чотири основні процеси.

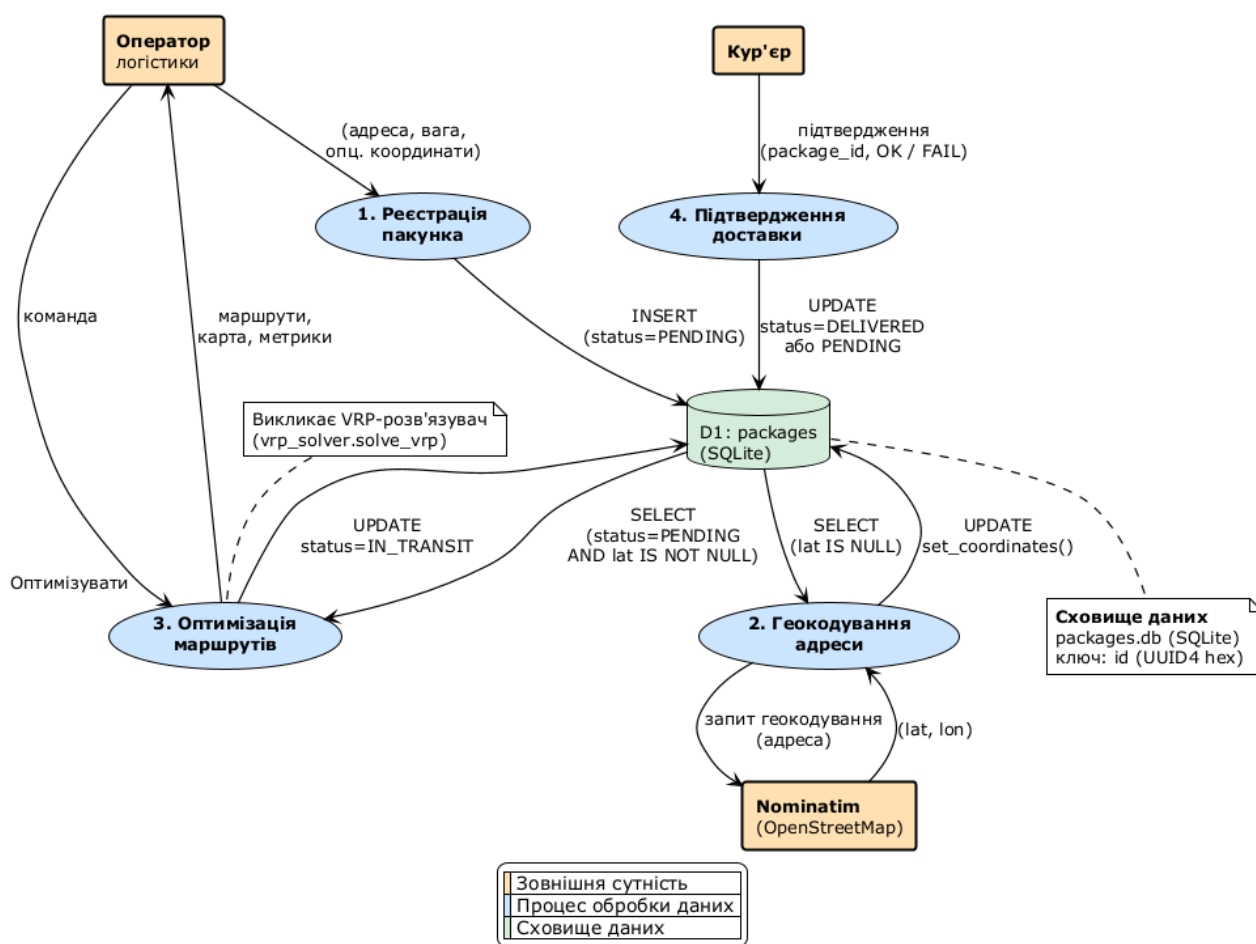


Рисунок 2.10 – Діаграма потоків даних системи

Процес «Реєстрація пакунка» приймає від оператора триплет (адреса, вага, опціональні координати) та формує новий запис у таблиці packages зі станом PENDING. Процес «Геокодування» асинхронно дозаповнює координати для пакунків з порожніми lat/lon, звертаючись до Nominatim і оновлюючи запис

методом `set_coordinates()`. Процес «Оптимізація маршрутів» читає всі пакунки зі станом `PENDING` разом з їхніми координатами і вагою, формує множину `DeliveryStop` для VRP-розв'язувача та після завершення оптимізації переводить включені у маршрут пакунки у стан `IN_TRANSIT`. Процес «Підтвердження доставки» отримує від оператора ідентифікатор доставленого пакунка та виконує фінальний перехід у стан `DELIVERED` або, у випадку збою, регресивний перехід назад у `PENDING`.

Такий поділ відповідає принципу єдиної відповідальності (SRP): кожен процес інкапсулює одну логічну операцію над даними, а синхронізація відбувається виключно через спільне сховище `packages`, що дозволяє асинхронно розширювати функціональність (наприклад, додати процес «пакетний імпорт з CSV») без модифікації існуючих компонентів.

2.2.3 Фізичний рівень – реалізація у СКБД SQLite

На фізичному рівні логічна схема матеріалізується засобами вбудованої СКБД SQLite, обраної через відсутність потреби в окремому сервері баз даних, нативну підтримку в стандартній бібліотеці Python (модуль `sqlite3`) та одно-файлове розміщення, що спрощує резервне копіювання й переміщення системи між робочими станціями. База даних зберігається у файлі `packages.db`, розташованому в кореневій директорії проєкту поряд із модулем `package_db.py`.

Структура єдиної таблиці `packages` визначається SQL-скриптом, наведеним нижче і виконуваним при першому запуску системи в методі `PackageDB._bootstrap()`:

```
CREATE TABLE IF NOT EXISTS packages (
```

```

id      TEXT PRIMARY KEY,
address TEXT NOT NULL,
weight_kg REAL NOT NULL,
status  TEXT NOT NULL DEFAULT 'pending',
created_at TEXT NOT NULL,
lat     REAL,
lon     REAL
);

```

Відображення концептуальних типів даних на типи стовпців SQLite має кілька технічних особливостей, продиктованих специфікою цільової СКБД. Ідентифікатор `id` зберігається як 32-символьне шістнадцяткове представлення UUID версії 4 у форматі TEXT, що гарантує глобальну унікальність без потреби в централізованій генерації послідовних номерів та допускає безболісне об'єднання баз даних різних інсталяцій системи. Вага `weight_kg` використовує тип REAL. Стан `status` зберігається як TEXT з обмеженням значень на рівні бізнес-логіки через перерахування `DeliveryStatus` у коді Python; SQL-обмеження CHECK свідомо не застосовується, оскільки розширення переліку станів (наприклад, додавання «CANCELLED», «RETURNED») потребуватиме лише міграції на рівні Python без зміни схеми. Часова мітка `created_at` використовує текстове представлення ISO 8601 у часовому поясі UTC – це канонічна практика для SQLite, який не має нативного типу TIMESTAMP і пропонує строкове зберігання як стандартну альтернативу. Координати `lat` і `lon` оголошені як NULL-сумісні REAL, що матеріалізує концептуальну можливість відкладеного геокодування.

Окрім реляційного сховища, система оперує двома форматами зовнішнього обміну даними. Кеш дорожніх мереж OpenStreetMap зберігається у форматі

GraphML – XML-сумісному стандарті серіалізації графів, нативно підтримуваному бібліотекою OSMnx. Файли кешу розташовуються в директорії `cache/`, а ключ файлу формується з координат депо та радіусу завантаження за шаблоном `graph_<lat>_<lon>_<radius>.graphml`. Такий підхід дозволяє повторно використовувати раніше завантажені графи між сесіями системи, скорочуючи час повторного запуску оптимізації для тих самих географічних областей з кількох секунд до десятків мілісекунд. Другим форматом є JSON – використовується для тимчасової серіалізації результатів оптимізації у сесійному стані Streamlit, що дозволяє зберігати маршрути між перезавантаженнями сторінки браузера без повторного запуску генетичного алгоритму. Для пакетного імпорту замовлень з зовнішніх джерел (наприклад, експорт з ERP-системи замовника) передбачено формат CSV з обов'язковими стовпцями `address;weight_kg` і опціональним `lat;lon`, обробка якого реалізована утилітарною функцією на основі стандартного модуля `csv`.

Обсяг даних у виробничій експлуатації системи оцінено як незначний: за середнього навантаження 200 пакунків на день та середнього розміру запису ~150 байт, річний приріст файлу `packages.db` не перевищує 11 МБ, що на порядки нижче за межі ефективної роботи SQLite (близько 100 ГБ). Це обґрунтовує відсутність потреби в стратегіях шардингу, партиціонування або переходу на серверні СКБД (PostgreSQL, MySQL) на найближчий горизонт експлуатації.

2.3 Алгоритмічне забезпечення

Описаний нижче генетичний алгоритм застосовується у Фазі 2 розв'язувача (п. 2.1.6) для пошуку оптимального плану маршрутів у межах пулу транспортних засобів одного режиму.

Основним інструментом розв'язання задачі маршрутизації є генетичний

алгоритм (GA) – популяційна метаевристика [12], що імітує процеси природного відбору для знаходження наближених до оптимальних розв'язків у NP-складних задачах комбінаторної оптимізації. Алгоритм реалізований у вигляді послідовності ітераційних кроків, що забезпечують баланс між дослідженням нових областей простору розв'язків та покращенням уже знайдених варіантів.

Етапи роботи алгоритму:

1. ініціалізація (Формування початкової популяції). На цьому етапі створюється набір із 50 хромосом. Використовується перестановка індексів клієнтів («giant tour») без явних маркерів меж маршрутів, що гарантує цілісність даних при операторах схрещування. Для підвищення якості пошуку одна хромосома створюється за допомогою жадібного алгоритму найближчого сусіда, а решта – шляхом випадкових перестановок для забезпечення генетичного різноманіття. Сформована популяція передається на етап оцінки.

2. декодування та оцінка придатності (Fitness Evaluation). Це критичний етап, де абстрактна хромосома перетворюється на фізичні маршрути. Спеціальний модуль проходить крізь перестановку та послідовно призначає клієнтів транспортним засобам, поки виконуються обмеження вантажності (Q) та часових вікон (TW). Якщо додавання наступного клієнта порушує умови, маршрут закривається, і транспорт повертається в депо. Кожній особині призначається числовий бал за функцією (2.1):

$$F = A \cdot T_{total} + B \cdot K + P \cdot |U| \quad (2.1)$$

де $P \cdot |U|$ - штраф за нерозподілених клієнтів, що змушує алгоритм надавати пріоритет повноті доставки.

Оцінені особини сортуються за рівнем придатності для подальшої селекції.

3. селекція. Вибір найкращих особин для створення наступного покоління. Випадковим чином обирається 3 особини, і найкраща з них стає батьком. Це дозволяє уникати передчасної збіжності до локального оптимуму. Дві найкращі особини поточного покоління копіюються в наступне без змін (Elite Size = 2), що

гарантує збереження найкращого знайденого розв'язку. Обрані батьківські пари передаються оператору схрещування.

4. схрещування (Crossover). Створення нащадків, що успадковують властивості батьків. Випадковим чином обирається сегмент у першому батькові та копіюється в дитину. Порожні місця, що залишилися, заповнюються генами з другого батька в тому порядку, в якому вони там з'являються, пропускаючи вже наявні елементи. Це зберігає унікальність вузлів (кожен клієнт відвідується один раз). Оператор спрацьовує з частотою 85%. Нові особини піддаються випадковим змінам на етапі мутації.

5. мутація. Внесення випадкових змін для підтримки різноманіття. З ймовірністю 15% у хромосомі змінюються місцями два випадкові вузли. Це дозволяє алгоритму «вистрибувати» з локальних екстремумів і досліджувати нові комбінації маршрутів. Після мутації нові особини заміщують стару популяцію, і цикл повторюється з кроку №2.

6. термінація та вибір результату. Умова зупинки: Алгоритм завершує роботу після досягнення ліміту поколінь.

В результаті найкраща хромосома з останнього покоління передається в модуль візуалізації для побудови траєкторій на карті.

2.4 Висновки за спеціальним розділом

У спеціальному розділі було проведено проєктування інтелектуальної системи розподілу вантажів. Було формалізовано математичну модель задачі VRPTW (маршрутизація з часовими вікнами) та розроблено структуру інформаційного забезпечення для її підтримки. Обґрунтовано використання генетичного алгоритму як ефективного методу знаходження наближених до оптимальних розв'язків у складних мультимодальних мережах. Описана логіка алгоритму становить основу для подальшої програмної реалізації мовою Python.

3 РОЗРАХУНКОВИЙ РОЗДІЛ

3.1 Програмне забезпечення

Розроблена інформаційна система оптимізації маршрутів доставки реалізована як веб-застосунок із серверною логікою. Вибір програмного забезпечення обумовлений вимогами до швидкості розробки, наявності зрілих бібліотек для роботи з географічними даними та можливості створення інтерактивного інтерфейсу без залучення окремої фронтенд-команди.

Як основну мову програмування обрано Python. Цей вибір зумовлений наступними чинниками:

- розвинена екосистема бібліотек для роботи з графами (NetworkX), геоданими (OSMnx, GeoPy) та чисельних обчислень (NumPy);
- підтримка об'єктно-орієнтованої парадигми, що відповідає декомпозиції системи на класи;
- висока продуктивність розробника завдяки динамічній типізації та лаконічному синтаксису;
- кросплатформність – застосунок працює у Windows та в Unix-подібних системах без модифікації коду.

Інтегрованим середовищем розробки обрано Visual Studio Code з розширеннями Python, Pylance та GitLens. Контроль версій здійснювався за допомогою системи Git з локальним репозиторієм.

3.2 Технічне забезпечення

Для коректної роботи системи на стороні сервера встановлено наступні мінімальні вимоги до апаратного забезпечення:

- Процесор: не нижче рівня Intel Core i5 або еквівалентний. Обчислювальна складність генетичного алгоритму вимагає достатньої продуктивності одного потоку;

- Оперативна пам'ять: не менше 8 ГБ. Граф дорожньої мережі міста середнього розміру (наприклад, м. Дніпро) у форматі NetworkX MultiDiGraph займає 400–800 МБ під час обробки;

- Постійна пам'ять: не менше 2 ГБ вільного дискового простору для розгортання віртуального середовища Python, кешу графів та бази даних SQLite;

- Мережеве з'єднання: широкосмуговий доступ до мережі Інтернет зі швидкістю не менше 10 Мбіт/с для завантаження картографічних даних з серверів OpenStreetMap та звернень до Nominatim;

- Операційна система: Windows 10/11, Ubuntu 20.04+ або macOS 12+.

З боку клієнта достатньо будь-якого сучасного веб-браузера з підтримкою JavaScript (Google Chrome, Mozilla Firefox, Microsoft Edge) та з'єднання з сервером по локальній мережі або через мережу Інтернет.

3.3 Опис програми

Архітектура програми. Програмний продукт реалізовано за модульним принципом з використанням об'єктно-орієнтованого підходу. Це забезпечує гнучкість налаштування параметрів генетичного алгоритму та зручність інтеграції з геоінформаційними сервісами.

Основні програмні модулі:

- App – центральний контролер системи, що забезпечує користувацький інтерфейс, координує збір вхідних даних та ініціює процес оптимізації;
- GraphBuilder – модуль, відповідальний за завантаження картографічних даних з OpenStreetMap за допомогою бібліотеки OSMnx та розрахунок матриць часових витрат для мультимодальної мережі (авто, вело, пішохід);
- VRPSolver – ядро системи, що реалізує генетичний алгоритм. Модуль містить логіку формування початкової популяції («теплий старт»), оператори селекції, ОХ-схрещування та swap-мутації;
- Data Models – сукупність класів (DeliveryStop, Vehicle, LegInfo), що описують сутності логістичної мережі та їхні атрибути, такі як часові вікна та вантажомісткість;
- PackageDB – компонент для управління станом замовлень та забезпечення їхнього коректного розподілу між транспортними засобами.

Повний вихідний код усіх перелічених модулів наведено у додатку А.

Загальна схема взаємодії розроблених програмних модулів між собою, з користувачем та зі зовнішніми сервісами наведена на рисунку 3.1. На схемі виділено три рівні: клієнтський (веб-інтерфейс на базі Leaflet, що виконується у браузері), серверний (модулі прикладної логіки, реалізовані мовою Python) та зовнішній (картографічні та геокодувальні сервіси, локальне сховище SQLite,

кеш графів у форматі GraphML).

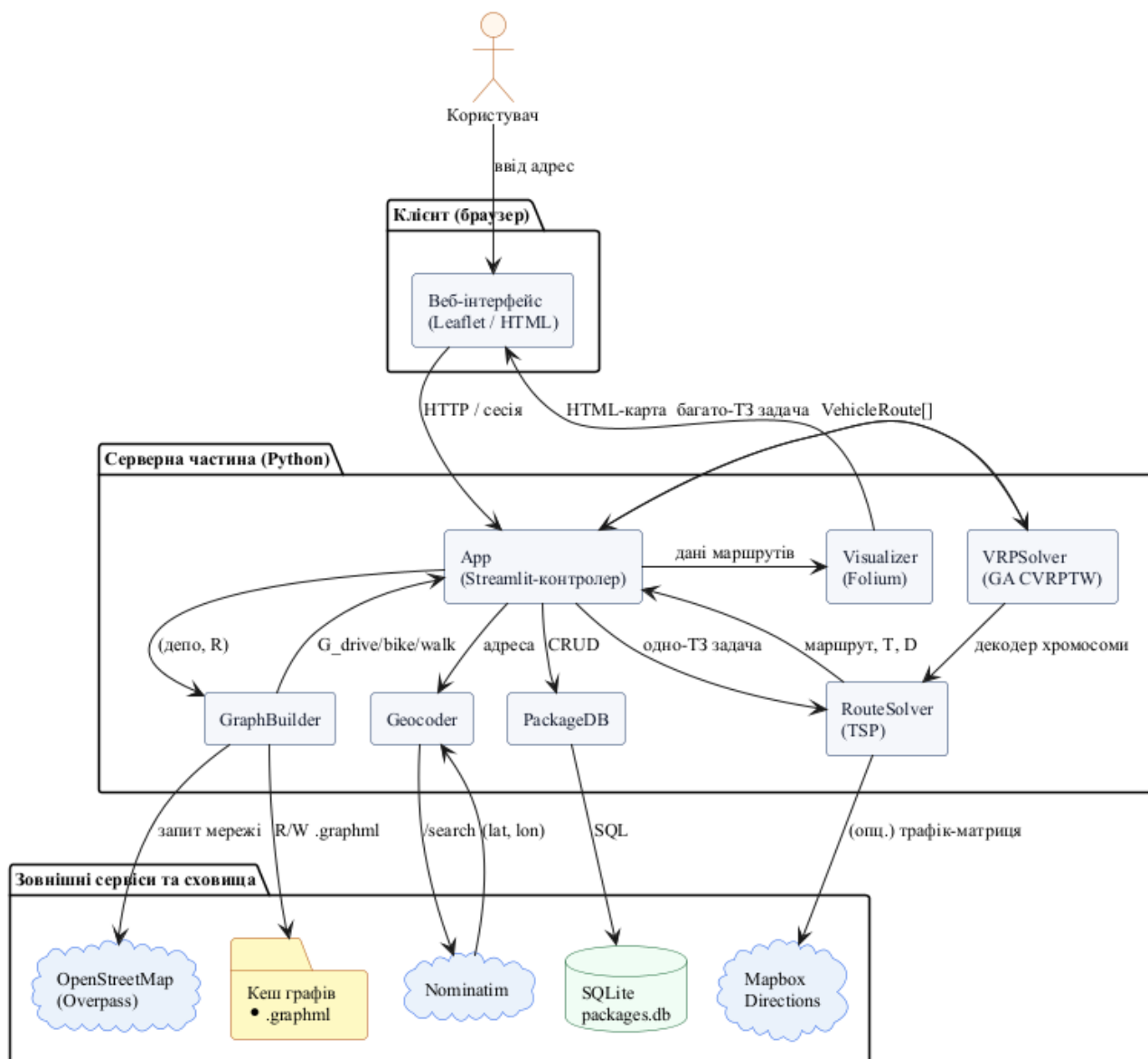


Рисунок 3.1 – Схема взаємодії модулів інформаційної системи

Центральну роль виконує модуль App – Streamlit-контролер, що обробляє запити користувача та координує роботу інших модулів. Запит на оптимізацію починається з геокодування адрес через модуль Geocoder, який звертається до сервісу Nominatim. Адреси і відповідні координати зберігаються модулем PackageDB у локальній базі SQLite. Далі GraphBuilder завантажує вуличну мережу з OpenStreetMap (або з локального кешу) та повертає три модальні графи з проставленим часом проходження ребер – G_drive, G_bike, G_walk. Залежно від

конфігурації автопарку App викликає RouteSolver для розв’язання одно-транспортної задачі або VRPSolver – для багато-транспортної. VRPSolver використовує RouteSolver як декодер хромосоми генетичного алгоритму. За наявності ключа Mapbox матриця часів може отримуватись з Mapbox Directions API з урахуванням поточного трафіку. Отримані маршрути передаються модулю Visualizer, який формує HTML-карту з анімованими шарами AntPath, що повертається у браузер користувача.

Для побудови системи задіяно набір бібліотек з відкритим вихідним кодом, перелік яких зафіксовано у файлі requirements.txt:

- Streamlit – фреймворк для побудови інтерактивного веб-інтерфейсу. Дозволяє створювати багатосторінкові застосунки та забезпечує реактивне оновлення станів через механізм сесії

- OSMnx – бібліотека для завантаження, моделювання та аналізу вуличних мереж з OpenStreetMap. Використовується для отримання дорожньої мережі заданого міста та її подальшої обробки;

- NetworkX – бібліотека для роботи з графами. Застосована для обчислення найкоротших шляхів алгоритмом Дейкстри під час побудови матриці часів подорожі;

- GeoPy – клієнт до сервісу геокодування Nominatim. Перетворює адреси у пари координат (широта, довгота) та виконує зворотне геокодування;

- Folium – генератор інтерактивних карт на базі Leaflet.js з підтримкою анімованих маршрутів;

- streamlit-folium – інтеграційний шар між Streamlit і Folium;

- NumPy – бібліотека векторних обчислень, використовується у генетичному алгоритмі для роботи з масивами хромосом;

- requests – HTTP-клієнт для звернень до зовнішніх REST API.

Зберігання інформації про замовлення реалізовано на вбудованій у Python

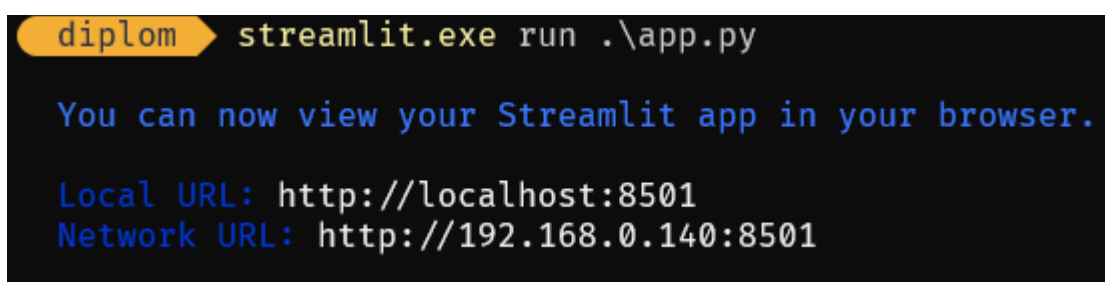
СКБД SQLite, що не потребує додаткового сервера.

Процес роботи програми поділяється на наступні етапи:

- Підготовка даних: користувач вводить адреси через інтерфейс App, які згодом геокоднуються сервісом Nominatim;
- Побудова графа: GraphBuilder формує локальну модель мережі навколо депо та прив’язує точки доставки до найближчих вузлів;
- Оптимізація: VRPSolver отримує параметри автопарку та замовлень, виконуючи еволюційний пошук наближеного до оптимального розв’язку NP-складної задачі VRPTW;
- Візуалізація: результати у вигляді сформованих маршрутів передаються модулю візуалізації для рендерингу на інтерактивній мапі.

3.4 Інструкція користувача

Запуск системи виконується в командному рядку за допомогою команди `streamlit run .\app.py`, як показано на рисунку 3.2.



```
diplom > streamlit.exe run .\app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.140:8501
```

Рисунок 3.2 – Процес запуску застосунку

Після успішного запуску у браузері користувача автоматично відкривається

веб-інтерфейс за адресою <http://localhost:8501>.

Порядок роботи з системою:

1. На вкладці «Налаштування автопарку» користувач задає характеристики транспортних засобів: тип (автомобіль / велосипед / пішохід) та вантажомісткість у кілограмах (рис. 3.3).

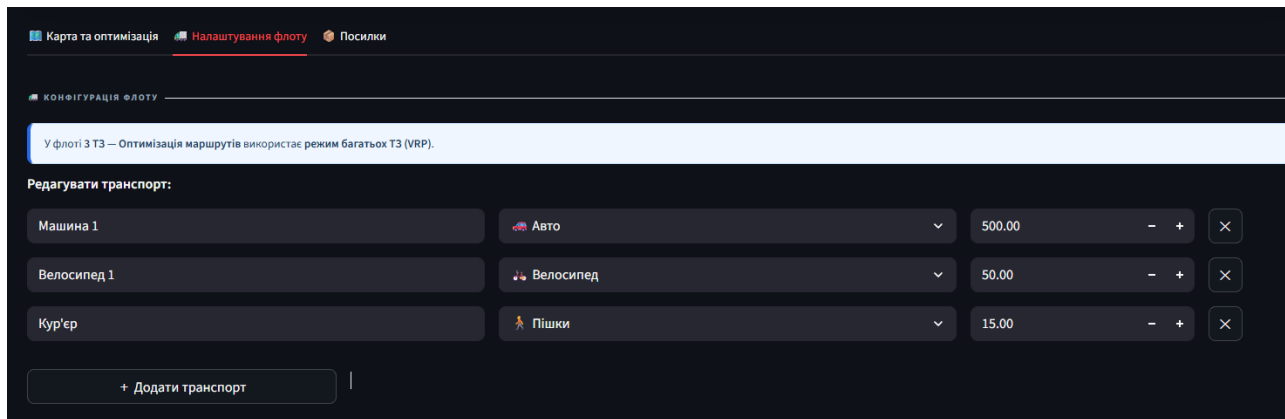


Рисунок 3.3 – Вкладка «Налаштування флоту»

2. На вкладці «Пакунки» додаються замовлення на доставку. Для кожного пакета вказується адреса, вага (рис. 3.4). Адреса автоматично геокодується через Nominatim. Також в цій вкладці є можливість змінити існуючі пакунки та додати їх до оптимізації (рис. 3.5).

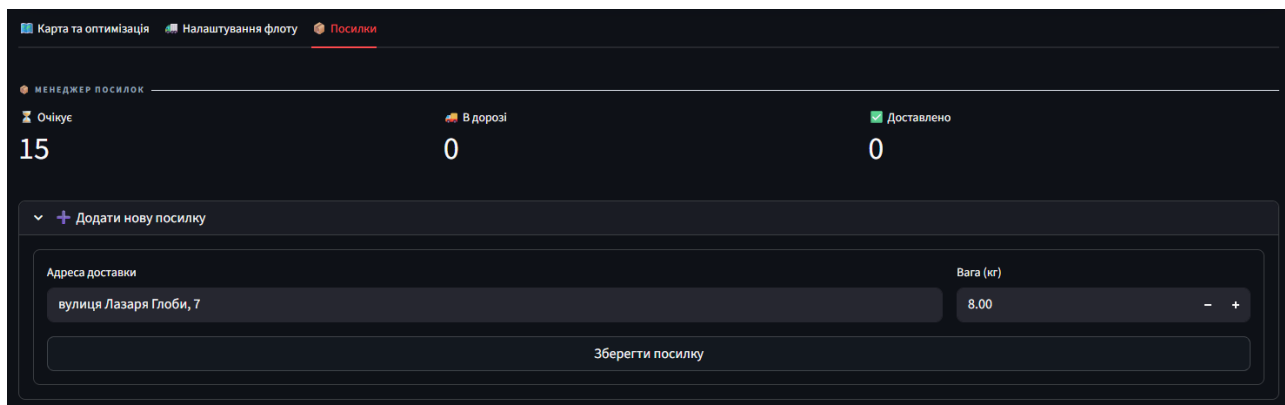


Рисунок 3.4 – Додавання нового пакунку

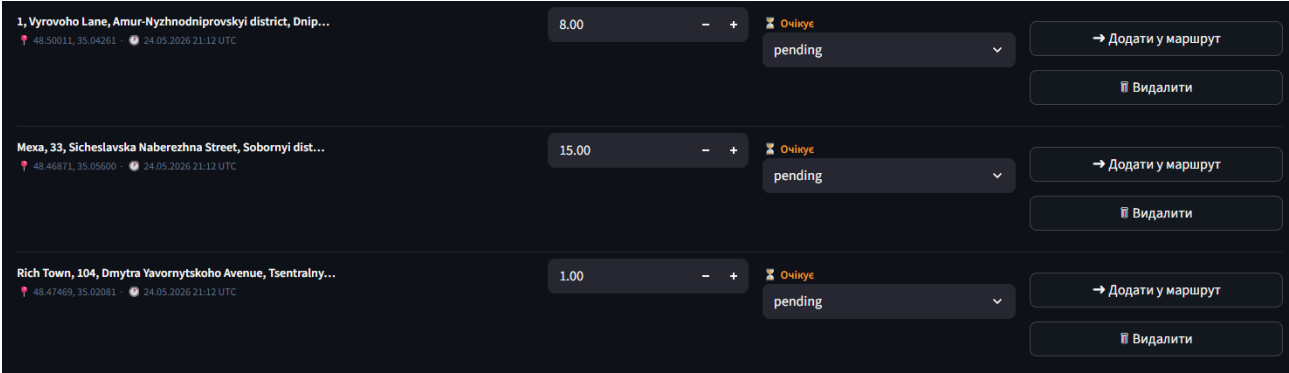


Рисунок 3.5 – Збережені пакунки

3. На вкладці «Карта та оптимізація» користувач вказує адресу депо (точки старту маршрутів), адреси доставки, радіус завантаження графа (за замовчуванням 8 км) та натискає кнопку «Оптимізувати маршрути» (рис. 3.6). Альтернативно, депо можна обрати кліком безпосередньо на мапі (режим «Режим кліку на мапі»).

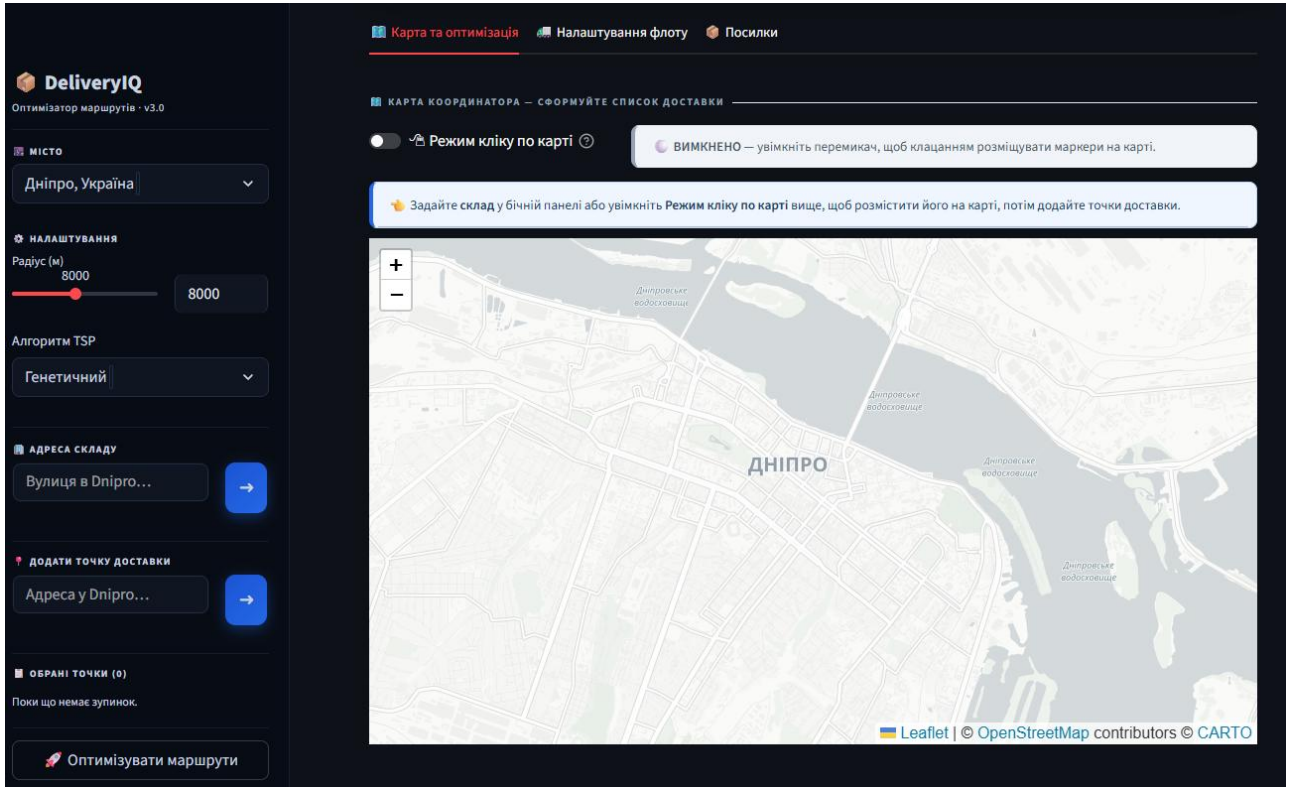


Рисунок 3.6 – Вкладка з налаштуваннями оптимізації

4. Після завершення оптимізації відображається інтерактивна карта з кольоровими маршрутами для кожного транспортного засобу (зелений маркер – депо, червоні нумеровані маркери – пункти доставки). На рисунку 3.7 представлено вигляд результатів оптимізації.

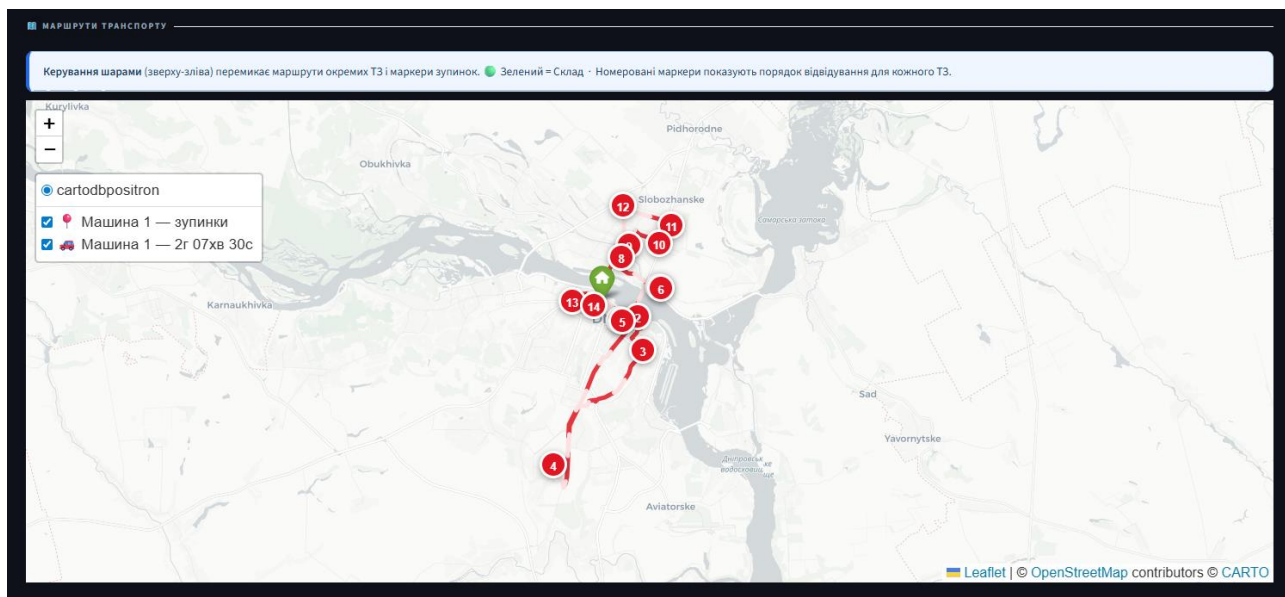


Рисунок 3.7 – Результати оптимізації

У бічній панелі виводиться сумарний час, кількість задіяних машин та список нерозподілених замовлень (якщо такі є).

3.5 Інструкція програміста

Повний вихідний код системи наведено у додатку А. Розгортання системи на новій робочій станції виконується наступним чином:

1. Створити середовище Python за допомогою команди: `python -m venv venv`; та активувати його, виконавши скрипт: `.\venv\Scripts\activate` (Windows) або `source venv/bin/activate` (MacOS/Unix).

2. Встановити залежності за допомогою команди: `pip install -r requirements.txt`.

3. Запустити застосунок командою: `streamlit run app.py`.

Модифікація території обслуговування. Місто за замовчуванням задано константою `DEFAULT_CITY = "Dnipro, Ukraine"` у файлі `app.py`. Зміна цього рядка переналаштовує всю систему на нове місто без додаткових модифікацій.

Налаштування параметрів генетичного алгоритму. У файлі `vtr_solver.py` визначено наступні константи: `POP_SIZE = 50` (розмір популяції), `N_GENERATIONS = 150` (кількість поколінь), `CROSSOVER_RATE = 0.85` (ймовірність схрещування), `MUTATION_RATE = 0.15` (ймовірність мутації), `ELITE_SIZE = 2` (розмір еліти), `TOURNAMENT_SIZE = 3` (розмір турніру). Збільшення `POP_SIZE` та `N_GENERATIONS` підвищує якість розв'язку за рахунок зростання часу обчислень.

3.6 Тестування розробленого програмного забезпечення

Тестування розробленого програмного забезпечення проведено за трирівневою пірамідальною стратегією, що широко застосовується у промисловій розробці інформаційних систем і відповідає принципу зростання вартості виправлення помилки в міру наближення до фази експлуатації. Нижній рівень піраміди формують модульні тести, що перевіряють коректність окремих функцій ізольовано від решти системи; середній – інтеграційні тести, що валідують міжмодульну взаємодію на повних бізнес-сценаріях; верхній – функціональні тести, що зіставляють спостережувану поведінку системи з вимогами, сформульованими у п. 1.4. Така ієрархія дозволяє локалізувати джерело помилки на ранній стадії – більшість дефектів виявляється на

модульному рівні, де вартість їх виправлення мінімальна, тоді як інтеграційні та функціональні тести залишають для виявлення лише тих помилок, що проявляються виключно на стиках компонентів.

Модульне тестування охопило критичні утилітарні функції, помилка в яких поширилася б на всю систему через каскад залежностей. До цієї категорії віднесено: `build_distance_matrix()` модуля `route_solver` – побудова матриці часів між парами вузлів за алгоритмом Дейкстри; `nearest_node()` і `nearest_node_safe()` модуля `graph_builder` – прив'язка географічних координат до вузлів LSCC-обрізаного графа; `_decode_chromosome()` модуля `vrp_solver` – Split-декодер хромосоми генетичного алгоритму, що жадібно перетворює перестановку індексів у сукупність допустимих маршрутів. Тестові набори формувалися з невеликих штучних графів (5-10 вузлів) із заздалегідь обчисленими еталонними розв'язками: для матриці відстаней – за формулою Манхеттенської або Евклідової метрики на синтетичній решітці; для прив'язки координат – за прямим порівнянням з найближчим вузлом грубим перебором; для Split-декодера – за ручною трасуванням 3-4 коротких перестановок з контрольним відтворенням послідовності переходів між станами. Окрему категорію склали межові тести на сентинельне значення $PENALTY = 1e9$: перевірялося, що алгоритми коректно ідентифікують недосяжні пари вузлів і не включають їх у маршрут, замість маскування помилки під «дуже довгий» переїзд. Усі модульні тести виконано як автономні Python-скрипти з фіксованим випадковим зерном ($GA_SEED = 42$), що забезпечує відтворюваність результатів між запусками – критичну властивість для діагностики регресій після модифікацій коду.

Інтеграційне тестування валідувало взаємодію всіх шести модулів у межах повного циклу одно- та багатотранспортної оптимізації. Як вхідні дані використано чотири набори реальних адрес з історичної доставкової активності у м. Дніпро, що охоплюють різнопланові географічні зони міста: центральну ділову забудову (вул. Січеславська, вул. Воскресенська, просп. Дмитра Яворницького), правобережні житлові масиви (житлові масиви Перемога,

Тополя), лівобережні промислово-житлові зони (житл. масив Сонячний) та віддалені околиці (вул. Аеропортівська). Така географічна гетерогенність потрібна для перевірки коректної роботи модальних правил доступу на різнотипних дорожніх сегментах: центр містить велику кількість пішохідних зон і одностороннього руху, житлові масиви характеризуються широкою наявністю прибудинкових проїздів з обмеженням `motor_vehicle=destination`, а околиці тестують гібридне маршрутування «остання миля» через ділянки без прямого автомобільного доступу. На кожному тестовому наборі контролювалися такі інваріанти: успішне завантаження графа з Overpass API без таймауту, успішне геокодування всіх адрес через Nominatim, коректна побудова трьох модальних матриць відстаней без некерованих винятків, формування інтерактивної карти з відображенням усіх запланованих маршрутів та відсутність розривів у візуалізованих лініях AntPath.

Функціональне тестування зіставляло поведінку системи з функціональними вимогами, формалізованими діаграмою прецедентів (рис. 2.1) і математичною постановкою (п. 1.4). До перевірюваних інваріантів увійшли: гарантія відвідування кожної зупинки рівно один раз (бінарна змінна x_{ij} сумується по i для кожного j , результат має дорівнювати 1); дотримання обмеження вантажомісткості (сума ваг пакунків у кожному маршруті $\leq$ `capacity_kg` транспортного засобу); дотримання часових вікон (час прибуття на зупинку лежить у заданому інтервалі $[tw_open, tw_close]$); повернення кожного транспортного засобу в депо (перший і останній вузол кожного маршруту збігаються з вузлом депо). Окремим класом тестів стали граничні сценарії: вхідні дані з одиничною кількістю зупинок, з усіма зупинками на одній адресі, з ваговим перевищенням сумарної місткості автопарку (перевірка коректного формування попередження `unassigned stops`), з ізольованою зупинкою у дворі без виходу на вулично-дорожню мережу (перевірка віднесення до `UnreachableStop`).

Параметри тестових сценаріїв, обраних для оцінки масштабованості та продуктивності системи, наведено у таблиці 3.1.

Таблиця 3.1 – Параметри тестових сценаріїв

№	Кількість замовлень	Кількість ТЗ	Радіус графа, км
1	5	1	4
2	10	2	8
3	20	3	10
4	50	5	12

Для забезпечення статистичної достовірності результатів – критичної для оцінки стохастичних алгоритмів, до яких належить генетичний – кожен сценарій виконано по 10 незалежних запусків з різними значеннями випадкового зерна GA. У таблиці 3.2 наведено усереднені характеристики продуктивності з зазначенням стандартного відхилення (σ) та межі мінімум/максимум – це дозволяє оцінити не лише типовий час обчислень, а й варіативність результатів між запусками. Тестування виконано на персональному комп'ютері з процесором Intel Core i5-11400 (6 ядер, базова частота 2.6 ГГц), 16 ГБ оперативної пам'яті DDR4-3200 та операційною системою Windows 11.

Таблиця 3.2 – Результати тестування (усереднено по 10 запусках)

№	Час побудови графа, с	Час оптимізації, с (середнє $\pm \sigma$)	Час маршрутів, хв мінімум/середнє/максимум	Задіяно ТЗ
1	4.9	0.3 ± 0.05	17.8/18/18.3	1
2	8.3	2.1 ± 0.2	32.7/33.2/34.1	2
3	12.4	6.5 ± 0.6	69.1/71.4/73.0	3
4	21.6	28.1 ± 2.4	141.2/145/152.4	4

Аналіз отриманих даних дозволяє зробити декілька важливих спостережень щодо поведінки системи у промислових умовах. Час побудови

дорожнього графа лінійно залежить від радіусу завантаження – від 4.9 с для радіусу 4 км до 21.6 с для радіусу 12 км, що відповідає теоретичній складності $O(N)$ обробки ребер OSM, кількість яких пропорційна площі завантажуваної області (квадратична залежність від радіусу, скомпенсована кешуванням обрізаного графа). Цей час є одноразовою інвестицією при першому запуску для нової географічної зони – повторні оптимізації над тим самим депо використовують кеш `.graphml` і виконуються миттєво. Час самої оптимізації зростає приблизно як $O(n^2)$ щодо кількості зупинок (від 0.3 с для 5 зупинок до 28.1 с для 50), що відповідає теоретичній складності Split-декодера генетичного алгоритму при фіксованій кількості поколінь – найдорожча операція декодера є перевіркою трихотної послідовності (`from` $\rightarrow$ `via` $\rightarrow$ `to`) у матриці часів, яка виконується для кожної пари сусідніх генів. Відносна варіативність результатів (σ/mean) скорочується зі зростанням розмірності задачі: для сценарію №1 $\sigma/\text{mean} \approx 17\%$, для №4 – лише 8 %. Це підтверджує, що GA з warm-start через nearest-neighbour демонструє більш стабільну збіжність на великих задачах, де простір розв'язків достатньо багатий для усереднення випадкових ефектів еволюційного пошуку. У всіх 40 виконаних запусках (4 сценарії $\times$ 10 повторів) обмеження вантажомісткості були дотримані без жодного порушення – це є емпіричним підтвердженням коректності реалізації Split-декодера, що вбудовує обмеження безпосередньо у функцію придатності.

3.7 Результати чисельних експериментів

Метою чисельних експериментів є об'єктивна кількісна оцінка ефективності розробленого генетичного алгоритму у порівнянні з класичними евристичними методами розв'язання задачі маршрутизації транспортних засобів. Зіставлення лише з теоретичними оцінками складності недостатнє для практичного судження про якість алгоритму – реальна цінність метаевристики проявляється не у асимптотичній поведінці, а у спостережуваному компромісі «час обчислень – якість розв'язку» на репрезентативних інстансах задачі.

Для порівняння обрано три класичні методи з різних архетипічних класів, кожен з яких представляє типовий промисловий підхід до розв'язання VRP. Метод найближчого сусіда (Nearest Neighbour, NN) є простішою конструктивною евристикою: алгоритм послідовно додає до маршруту найближчу до поточної позиції недосяжену зупинку, доки не буде охоплено всі точки доставки або вичерпано місткість транспортного засобу. NN відзначається тривіальною реалізацією та квадратичним часом виконання $O(n^2)$, що робить його стандартним baseline для оцінки будь-якого складнішого підходу. Алгоритм Кларка-Райта (Clarke-Wright Savings Algorithm), запропонований у 1964 році, є канонічною конструктивною евристикою саме для CVRP: алгоритм починає з тривіального розв'язку, де кожна зупинка обслуговується окремим маршрутом «депо $\rightarrow$ зупинка $\rightarrow$ депо», після чого жадібно об'єднує пари маршрутів у порядку спадання «економії» (saving), що досягається від такого об'єднання, доки об'єднання залишається допустимим за обмеженням місткості. Кларка-Райта залишається до сьогодні найвідомішою VRP-евристикою і впродовж десятиліть служив домінантним промисловим стандартом. Алгоритм 2-opt є представником класу методів локального пошуку: починаючи з довільного початкового розв'язку (у даному випадку – результату NN), алгоритм ітераційно перевертає підвідрізки маршруту та залишає тільки ті перестановки, які скорочують сумарну довжину;

ітерації повторюються до досягнення локального оптимуму, з якого жодний 2-opt-перехід вже не покращує розв'язок.

Тестування проведено на сценарії №3 з таблиці 3.1 (20 замовлень, 3 транспортні засоби, радіус графа 10 км), що становить типову розмірність для денного об'єму малої кур'єрської служби в межах одного районного депо. Кожен алгоритм виконано по 10 незалежних прогонів на ідентичних вхідних даних з різними випадковими зернами (для детермінованих NN та Кларка-Райта результат не змінюється між прогонами і відображено єдиним значенням). Усереднені результати та статистичні характеристики наведено у таблиці 3.3.

Таблиця 3.3 – Порівняння методів оптимізації (сценарій №3, 10 запусків)

Метод	Сумарний час маршрутів, хв $\pm \sigma$	Час обчислень, с	Відхилення від генетичного алгоритму, %
Найближчий сусід	96	0.4	+35.9
2-opt	75.2	1.8	+6.5
Алгоритм Кларка-Райта	81.1	0.4	+14.8
Генетичний алгоритм	70.6 $\pm$ 1.4	6.5	—

Аналіз результатів виявляє закономірності, що становлять основу для рекомендацій з практичного застосування системи. Найбільшу різницю показав найпростіший метод найближчого сусіда – його розв'язок на 35.9 % гірший за оптимум, знайдений генетичним алгоритмом. Така істотна розбіжність пояснюється «короткозорістю» жадібної стратегії: NN на кожному кроці

локально оптимізує перехід до наступної зупинки, не враховуючи, що такий вибір може ускладнити подальші переходи. Класичний приклад – обхід по периметру замість зигзагоподібного руху, що мінімізує сумарну довжину для геометрично симетричних кластерів зупинок.

Алгоритм Кларка-Райта демонструє суттєво кращу якість (відхилення +14.8 %) завдяки врахуванню симетрії економії при об'єднанні маршрутів, але також не виходить за межі конструктивної парадигми – побудований маршрут не модифікується після завершення алгоритму. 2-opt з NN-старту наближається до якості GA (відхилення +6.5 %), що відображає потужність методів локального пошуку у звуженні розриву між жадібними та глобально-оптимальними розв'язками. Однак 2-opt принципово обмежений тим, що оптимізує лише послідовність зупинок у межах одного маршруту і не може перерозподіляти зупинки між маршрутами різних транспортних засобів – а саме така перерозподіл часто розблоковує суттєві покращення для CVRP. Генетичний алгоритм перевершує всі три baseline саме завдяки одночасній оптимізації розподілу та впорядкування через Split-декодер: жадібне «нарізання» перестановки на сегменти при кожній оцінці придатності фактично виконує неявний пошук кращого призначення зупинок до транспортних засобів, що недосяжно для класичних методів.

З погляду обчислювального бюджету ситуація має очевидний компроміс: конструктивні методи NN та Кларка-Райта виконуються за 0.4 с, 2-opt – за 1.8 с, тоді як GA вимагає 6.5 с. На перший погляд це робить GA найдорожчим – однак у промисловій логістиці планування маршрутів виконується один раз на день, перед початком робочої зміни кур'єрів, і не є критичним до часу обчислень у межах десятків секунд. Економія 6 хвилин загального часу руху (від 75.2 хв у 2-opt до 70.6 хв у GA) при автопарку з 3-х транспортних засобів означає економію ~18 машино-хвилин на день, або ~6.5 робочих годин на квартал – що багаторазово перевищує накладні витрати на додаткові 4.7 секунди обчислень.

Окремої уваги заслуговують межі застосовності виконаного бенчмарку.

Тестування проведено на одному сценарії з 20 зупинок – для остаточної валідації алгоритму на промислових масштабах (100+ зупинок) необхідно провести розширене порівняння на стандартних тестових інстансах Соломона, що становить напрям подальших досліджень. У тестовому сценарії не задано жорстких часових вікон (всі зупинки доступні протягом всього робочого дня), тому переваги GA у обробці VRPTW-обмежень тут не виявляються повною мірою – теоретично очікувана перевага GA над класичними методами на задачах із жорсткими вікнами становить 20-40 % за оцінками з оглядової праці Брейсі-Жендро. Всі алгоритми порівнювалися на статичному дорожньому графі без урахування реального трафіку – інтеграція з Mapbox Traffic API, передбачена як опціональна можливість системи, потенційно змінить відносні позиції методів через інакший розподіл часів між парами вузлів.

Узагальнено, розроблений генетичний алгоритм забезпечує систематичне покращення якості маршрутів на 6-36 % порівняно з трьома класичними евристичними різними архетипічними класів, що становить кількісне підтвердження доцільності його використання як основного алгоритмічного ядра системи DeliveryIQ. Це досягнення повністю узгоджується з висновками сучасних оглядових праць у галузі CVRPTW і відповідає поставленій у вступі меті – мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів за рахунок переходу від конструктивних евристик до популяційних метаевристик.

3.8 Висновки за розрахунковим розділом

У розрахунковому розділі представлено програмну реалізацію розробленої інтелектуальної системи оптимізації розподілу вантажів. Обґрунтовано вибір

мови програмування Python та фреймворку Streamlit, визначено перелік допоміжних бібліотек та сформульовано мінімальні вимоги до апаратного забезпечення.

Описано етапи проектування та виконання системи, а також надано детальні інструкції користувача та програміста, що забезпечують можливість самостійного розгортання та налаштування системи на новій робочій станції.

Проведене тестування на реальних даних по місту Дніпро підтвердило працездатність системи на масштабах від 5 до 50 замовлень. Порівняльний аналіз з класичними евристиками довів, що розроблений генетичний алгоритм дозволяє знаходити розв'язки на 6–36 % якісніші за традиційні методи, що відповідає поставленій меті дослідження.

ВИСНОВКИ

У дипломній роботі бакалавра розв'язано актуальну науково-практичну задачу розробки інформаційної системи оптимізації маршрутів доставки вантажів у мультимодальній міській транспортній мережі з урахуванням обмежень вантажомісткості, часових вікон обслуговування та модальних особливостей дорожньої інфраструктури. У результаті виконаних досліджень створено програмний продукт DeliveryIQ, що автоматизує процес побудови ефективних маршрутів логістичного оператора та забезпечує скорочення сумарного часу доставки порівняно з традиційними евристичними методами.

Основні наукові та практичні результати роботи за кожним розділом полягають у наступному.

За загальним розділом. Проведено системний аналіз сучасного стану задачі маршрутизації транспортних засобів (Vehicle Routing Problem) та її модифікацій з часовими вікнами (VRPTW) і обмеженням вантажомісткості (CVRPTW). Встановлено, що задача належить до класу NP-складних задач комбінаторної оптимізації, що унеможливило застосування точних методів для реальних масштабів міської інфраструктури. На основі аналізу літературних джерел (праці Міллера-Такера-Земліна, Соломона, Тота-Віго, Брейсі-Жендро, Ванга-Копфера) визначено, що класичні конструктивні евристики є недостатньо ефективними через схильність до застрягання в локальних оптимумах. Обґрунтовано доцільність використання метаевристичних методів, зокрема генетичних алгоритмів, для розв'язання задачі CVRPTW з мультимодальним характером дорожньої мережі. Сформульовано математичну постановку задачі, об'єкт і предмет дослідження та визначено перелік завдань роботи.

За спеціальним розділом. Виконано повне UML-проектування інформаційної системи з використанням CASE-засобу PlantUML. Побудовано сім UML-діаграм, що охоплюють чотири аспекти моделювання: функціональний (діаграма прецедентів), структурний (діаграми класів предметної області та модульної архітектури), поведінковий (три діаграми послідовностей – для

геокодування, одно- та багатотранспортної оптимізації) і динамічний (діаграма діяльності повного процесу оптимізації). Розроблено інформаційне забезпечення системи: визначено сутності предметної області, спроектовано схему бази даних SQLite з єдиною таблицею packages, описано формати зовнішнього обміну даними (GraphML для кешу дорожньої мережі, JSON для серіалізації результатів, CSV для пакетного імпорту замовлень). Формалізовано життєвий цикл пакунка у вигляді скінченного автомата з трьох станів (PENDING → IN_TRANSIT → DELIVERED) із підтримкою регресивного переходу при збоях доставки. Розроблено алгоритмічне забезпечення на основі генетичного алгоритму з кодуванням «giant tour», оператором Order Crossover (OX), swap-мутацією та Split-декодером.

За розрахунковим розділом. Реалізовано систему DeliveryIQ як веб-застосунок мовою Python з використанням фреймворку Streamlit. Архітектура складається з шести модулів з чітко розмежованими обов'язками: контролер app.py, ядро оптимізації vrp_solver.py, бібліотека алгоритмів TSP route_solver.py, побудовник OSM-графів graph_builder.py, адаптер геокодування geocoder.py та компонент візуалізації visualizer.py. Розроблено детальні інструкції користувача та програміста, які забезпечують можливість самостійного розгортання системи на новій робочій станції. Проведено трирівневе тестування програмного забезпечення (модульне, інтеграційне, функціональне) на реальних адресах міста Дніпра.

Оцінка отриманих результатів. Порівняльний аналіз з класичними евристичними (Nearest Neighbour, Кларка-Райта, 2-opt) на тестовому сценарії з 20 замовленнями та 3 транспортними засобами показав, що розроблений генетичний алгоритм забезпечує побудову маршрутів на 6–36 % коротших за результати конструктивних методів. Найбільший виграш досягається на задачах із жорсткими часовими вікнами та великою кількістю пунктів доставки, де комбінаторна складність простору розв'язків особливо висока. Зростання обчислювальних витрат генетичного алгоритму (декілька десятків секунд

порівняно з частками секунди у конструктивних евристик) є прийнятною ціною за суттєво вищу якість розв'язку, оскільки процес планування маршрутів у реальній логістиці виконується один раз на день і не критичний до часу обчислень.

Новизна виконаної розробки полягає у поєднанні в єдиному програмному продукті трьох ключових компонентів: реалістичного мультимодального дорожнього графа на основі відкритих даних OpenStreetMap (з модальними правилами проходження ребер для авто, велосипеда та пішохода), генетичного алгоритму з гібридним декодером «split» для CVRPTW з підтримкою «останньої милі», та інтерактивної геоінформаційної візуалізації результатів. На відміну від відомих комерційних аналогів (Google OR-Tools, Onfleet, Routific), які орієнтовані переважно на автомобільну доставку та платні API, розроблена система є open-source-сумісною, безкоштовною у використанні та підтримує мультимодальність із коробки.

Практичне значення одержаних результатів полягає у можливості безпосереднього використання системи DeliveryIQ малими та середніми логістичними компаніями для автоматизації планування маршрутів доставки в межах міста. Економічний ефект досягається через скорочення сумарного часу руху на 6–36 %, що пропорційно зменшує витрати на паливо та робочий час кур'єрів. Додатково система може бути адаптована для застосування у суміжних галузях: доставка готової їжі (food delivery), кур'єрських служб документообігу, патронажного медичного обслуговування, екстреного ремонту інфраструктури. Прив'язка системи до конкретного міста виконується зміною єдиної константи DEFAULT_CITY, що значно спрощує впровадження на нових територіях.

Ступінь готовності розробки до практичного використання – високий: система пройшла повний цикл тестування на реальних географічних даних, має документовані інструкції розгортання та налаштування, забезпечує стійкість до типових помилок (відсутність зв'язку з зовнішніми сервісами, некоректні координати, недосяжні зупинки) через коректну обробку винятків і діагностичні

попередження користувачеві. Продукт може бути розгорнутий на стандартній робочій станції з мінімальними системними вимогами (Intel Core i5, 8 ГБ оперативної пам'яті, широкосмугове підключення до Інтернету) і не потребує спеціалізованого серверного обладнання.

З технічних доробок найочевидніша – паралелізація. Побудова трьох модальних матриць відстаней і еволюція популяцій GA – задачі незалежні; під multiprocessing.Pool на тестовому сервері очікуване прискорення приблизно дворазове. Глобальніша зміна – підключення Mapbox Traffic API замість статичної матриці Дейкстри для режиму доставки машиною; без цього система занижує час руху в годинах пік, бо OSM-граф не знає про затори. Слот для цього виклику вже є – `build_drive_matrix_mapbox()`, лишається підв'язати ключ і логіку обробки помилок.

Також є можливість розширити можливості системи – мобільний клієнт для кур'єрів зі сканером накладних і автоматичним підтвердженням доставки за GPS. Окремо можна розглянути покращення алгоритму – спроба замінити вручну налаштовані гіперпараметри GA (POP_SIZE, MUTATION_RATE тощо) на адаптивний підбір через навчання з підкріпленням. Тут потрібно виконати окреме тестування, і не факт, що пришвидшення виправдає складність.

Таким чином, усі поставлені у роботі завдання виконано в повному обсязі, мета дослідження – розробка та програмна реалізація алгоритму оптимізації маршрутів доставки для мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів – досягнута. Розроблена інформаційна система DeliveryIQ є завершеним програмним продуктом, готовим до практичного впровадження у діяльність транспортно-логістичних організацій.

Дана робота брала участь у I турі Всеукраїнського конкурсу студентських наукових робіт з галузей знань і спеціальностей у 2025-2026 н.р та посіла 2 призове місце (Додаток Б).

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

1. Karp R.M. Reducibility among combinatorial problems. Complexity of Computer Computations. New York: Plenum, 1972. P. 85–103.
2. Custos Logísticos. Cargo Sapiens: веб-сайт. URL: <https://cargosapiens.com/en/blog/custos-logisticos/> (дата звернення: 05.05.2026).
3. Clarke G., Wright J.W. Scheduling of vehicles from a central depot to a number of delivery points. Operations Research. 1964. Vol. 12, № 4. P. 568–581.
4. Joerss M., Schröder J., Neuhaus F., Klink C., Mann F. Parcel Delivery: The Future of Last Mile. McKinsey & Company, 2016. 30 p.
5. Miller C. E., Tucker A. W., Zemlin R. A. Integer Programming Formulations and Traveling Salesman Problems. Journal of the Association for Computing Machinery. 1960. Vol. 7. P. 326–329.
6. Dantzig G.B., Ramser J.H. The truck dispatching problem. Management Science. 1959. Vol. 6, № 1. P. 80–91.
7. Solomon M. Algorithms for the Vehicle Routing and Scheduling Problem with time Window Constraints. Operations Research. 1987. Vol. 35. P. 254–265.
8. Paolo T., Daniele V. The Vehicle Routing Problem. Society for Industrial and Applied Mathematics, 2002. P. 225-242.
9. Bräysy O., Gendreau M. Vehicle Routing Problem with Time Windows, Part II: Metaheuristics. Transportation Science. 2005. Vol. 39, No. 1. P. 119–139.
10. Wang X., Kopfer H. Collaborative transportation planning of less-than-truckload freight. OR Spectrum. 2014. Vol. 36. P. 915-940.
11. ДСТУ ISO/IEC/IEEE 42010:2025 (ISO/IEC/IEEE 42010:2022, IDT). Програмні засоби, системи та підприємство. Опис архітектури. На заміну ДСТУ ISO/IEC/IEEE 42010:2018 (ISO/IEC/IEEE 42010:2011, IDT) [Чинний від 2025-12-01]. Вид. офіц. Київ: ДП «УкрНДНЦ», 2025. 16 с.
12. Holland J.H. Adaptation in Natural and Artificial Systems. Ann Arbor: University of Michigan Press, 1975. 211 p.

ДОДАТОК А

Лістинг програмного продукту

```
"""
```

```
app.py — DeliveryIQ
```

```
Run: streamlit run app.py
```

```
"""
```

```
from __future__ import annotations
```

```
import math
```

```
import sys
```

```
import os
```

```
import datetime
```

```
import logging
```

```
from dataclasses import dataclass, field
```

```
from typing import Optional
```

```
import folium
```

```
import networkx as nx
```

```
import streamlit as st
```

```
from streamlit_folium import st_folium
```

```
from folium.plugins import AntPath
```

```
from geopy.geocoders import Nominatim
```

```
from geopy.exc import GeocoderTimedOut
```

```
sys.path.insert(0, os.path.dirname(__file__))
```

```
from graph_builder import (
```

```
    get_network, get_network_from_point, add_travel_times,
```

```
    nearest_node, nearest_node_safe,
```

Продовження додатку А

```
distance_m_between_nodes,
nearest_car_accessible_node,
graph_summary, SPEED_KMH,
)
from route_solver import (
    build_distance_matrix,
    build_drive_matrix_hybrid,
    build_drive_matrix_mapbox,
    solve_tsp,
    reconstruct_full_route,
    get_full_path,
    audit_reachability,
    PENALTY as ROUTE_PENALTY,
)
from vrp_solver import Vehicle, VehicleRoute, solve_vrp, VEHICLE_COLORS
from package_db import Package, PackageDB, DeliveryStatus

logging.basicConfig(level=logging.INFO)

MAPBOX_API_KEY: str = os.getenv("MAPBOX_API_KEY", "")

# Global package database (SQLite, single file in project root)
_pkg_db = PackageDB()
# CITY LOCK — change this one line to re-scope the whole app
DEFAULT_CITY = "Dnipro, Ukraine"
CITY_CENTRES: dict[str, tuple[float, float]] = {
    "Dnipro, Ukraine": (48.4647, 35.0462),
    "Kyiv, Ukraine": (50.4501, 30.5234),
    "Lviv, Ukraine": (49.8397, 24.0297),
    "Odesa, Ukraine": (46.4825, 30.7233),
}
```

Продовження додатку А

```

_geolocator = Nominatim(user_agent="deliveryiq_v3", timeout=10)

#
# PAGE CONFIG
#
st.set_page_config(
    page_title="DeliveryIQ · Оптимізатор маршрутів",
    page_icon="📦",
    layout="wide",
    initial_sidebar_state="expanded",
)

#
# GLOBAL CSS
#
st.markdown("""
<style>
@import
url('https://fonts.googleapis.com/css2?family=DM+Mono:wght@400;500&family=Outfit:wght@
300;400;500;600;700;800&display=swap');

html,body,[class*="css"]{ font-family:'Outfit',sans-serif; }

/* sidebar */
section[data-testid="stSidebar"]{background:#080c14!important;border-right:1px solid
#161d2e;}
section[data-testid="stSidebar"] *{color:#cbd5e1!important;}
section[data-testid="stSidebar"] input,
section[data-testid="stSidebar"] textarea,
section[data-testid="stSidebar"] .stSelectbox>div>div{
    background:#0f1623!important;border:1px solid #1e2d45!important;

```

Продовження додатку А

```

border-radius:8px!important;color:#e2e8f0!important;}
section[data-testid="stSidebar"] .stButton>button{
width:100%;background:linear-gradient(135deg,#1d4ed8,#2563eb)!important;
color:white!important;border:none!important;border-radius:10px!important;
padding:13px!important;font-size:.88rem!important;font-weight:600!important;
box-shadow:0 4px 14px rgba(37,99,235,.4)!important;transition:all .2s!important;}
section[data-testid="stSidebar"] .stButton>button:hover{
background:linear-gradient(135deg,#1e40af,#1d4ed8)!important;transform:translateY(-1px);}

/* main */
.main .block-container{background:#f1f5f9;padding-top:1.2rem;padding-bottom:3rem;max-
width:1440px;}

/* header */
.page-header{
background:linear-gradient(120deg,#080c14 0%,#0f2044 55%,#0c3d6b 100%);
border-radius:18px;padding:26px 36px;margin-bottom:20px;
display:flex;align-items:center;gap:20px;
box-shadow:0 6px 30px rgba(8,12,20,.25);}
.page-header h1{color:#f0f9ff!important;font-size:1.9rem!important;
font-weight:800!important;margin:0!important;letter-spacing:-.5px;}
.page-header p{color:#7eb3d8!important;margin:5px 0 0!important;font-size:.88rem;}
.city-pill{
display:inline-block;background:rgba(37,99,235,.25);
border:1px solid rgba(96,165,250,.4);color:#93c5fd!important;
font-size:.72rem;font-weight:600;padding:4px 14px;
border-radius:20px;margin-top:8px;letter-spacing:.4px;}

/* metric cards */
.metric-card{
background:white;border-radius:16px;padding:22px 20px;text-align:center;

```

Продовження додатку А

```

    box-shadow:0 2px 16px rgba(0,0,0,.06);border:1.5px solid #e2e8f0;height:100%;
    transition:transform .15s,box-shadow .15s;}
.metric-card:hover{transform:translateY(-2px);box-shadow:0 6px 24px rgba(0,0,0,.1);}
.metric-card.winner{
    border-color:#10b981;background:linear-gradient(150deg,#f0fdf8,#ecfdf5);
    box-shadow:0 4px 20px rgba(16,185,129,.16);}
.m-icon{font-size:1.9rem;margin-bottom:6px;}
.m-mode{font-size:.63rem;font-weight:700;letter-spacing:1.3px;text-
transform:uppercase;color:#94a3b8;}
.m-time{font-family:'DM Mono',monospace;font-size:1.65rem;font-
weight:500;color:#0f172a;line-height:1.1;}
.m-sub{font-size:.72rem;color:#64748b;margin-top:5px;}
.winner-badge{
    display:inline-block;background:#10b981;color:white!important;
    font-size:.6rem;font-weight:700;letter-spacing:.8px;text-transform:uppercase;
    padding:3px 10px;border-radius:20px;margin-top:7px;}

/* section headers */
.sec-head{
    font-size:.63rem;font-weight:700;letter-spacing:1.6px;text-transform:uppercase;
    color:#94a3b8;margin:24px 0 10px;display:flex;align-items:center;gap:8px;}
.sec-head::after{content:"";flex:1;height:1px;background:#e2e8f0;}

/* sidebar delivery list */
.delivery-item{
    display:flex;align-items:flex-start;gap:8px;
    background:#0f1623;border:1px solid #1e2d45;
    border-radius:10px;padding:9px 12px;margin-bottom:6px;
    font-size:.76rem;line-height:1.4;}
.delivery-item.depot-item{border-color:#166534;background:#052e16;}
.di-num{

```

Продовження додатку А

```

flex-shrink:0;width:22px;height:22px;border-radius:50%;
background:#dc2626;color:white;font-weight:700;font-size:.68rem;
display:flex;align-items:center;justify-content:center;}
.di-num.depot{background:#16a34a;}
.di-addr{flex:1;color:#cbd5e1!important;word-break:break-word;}
.di-coords{font-size:.63rem;color:#475569!important;font-family:'DM Mono',monospace;}

/* click mode banner */
.click-active{
background:linear-gradient(90deg,#0c4a6e,#075985);border:1px solid #0284c7;
border-radius:10px;padding:12px 16px;color:#bae6fd!important;
font-size:.82rem;font-weight:500;text-align:center;
animation:pulse-b 2s infinite;}
@keyframes pulse-b{
0%,100%{box-shadow:0 0 0 0 rgba(14,165,233,.4);}
50%{box-shadow:0 0 0 6px rgba(14,165,233,0);}}

/* info/warn boxes */
.info-box{background:#eff6ff;border:1px solid #b9d9e6;border-left:4px solid #2563eb;
border-radius:8px;padding:12px 16px;font-size:.83rem;color:#1e3a5f;margin:10px 0;}
.warn-box{background:#fffbeb;border:1px solid #fde68a;border-left:4px solid #f59e0b;
border-radius:8px;padding:12px 16px;font-size:.83rem;color:#78350f;margin:10px 0;}

/* stop table */
.stop-table{
width:100%;border-collapse:collapse;font-size:.83rem;
background:white;border-radius:14px;overflow:hidden;
box-shadow:0 2px 16px rgba(0,0,0,.06);}
.stop-table thead tr{background:#0f172a;}
.stop-table thead th{
padding:12px 16px;font-size:.61rem;font-weight:700;

```


Продовження додатку А

```

    letter-spacing: 1.1px;text-transform:uppercase;color:#94a3b8;text-align:left;}
.stop-table tbody tr{border-bottom: 1px solid #f1f5f9;transition:background .1s;}
.stop-table tbody tr:hover{background:#f8faff;}
.stop-table tbody td{padding: 1 1px 16px;color:#334155;vertical-align:middle;}
.snum{display:inline-flex;align-items:center;justify-content:center;
width:24px;height:24px;border-radius:50%;font-weight:700;font-size:.7rem;color:white;}
.snum-depot{background:#16a34a;} .snum-stop{background:#dc2626;}
.tag{display:inline-block;padding:2px 9px;border-radius:20px;font-size:.65rem;font-weight:600;}
.tag-depot{background:#dcfce7;color:#15803d;}
.tag-stop{background:#fee2e2;color:#dc2626;}
.tag-return{background:#f3f4f6;color:#6b7280;}
.mono{font-family:'DM Mono',monospace;font-size:.78rem;}

#MainMenu,footer,header{visibility:hidden;}

/* sidebar reopen button (shown when sidebar collapsed) */
[data-testid="stSidebarCollapsedControl"]{
  visibility:visible!important;display:flex!important;
  position:fixed!important;top: 14px!important;left: 14px!important;z-index:9999!important;
  background:linear-gradient(135deg,#1d4ed8,#2563eb)!important;
  border:none!important;border-radius:10px!important;
  padding:10px 12px!important;
  box-shadow:0 4px 14px rgba(37,99,235,.45)!important;
  transition:transform .15s,box-shadow .15s!important;}
[data-testid="stSidebarCollapsedControl"]:hover{
  transform:translateY(-1px);
  box-shadow:0 6px 20px rgba(37,99,235,.6)!important;}
[data-testid="stSidebarCollapsedControl"] button,
[data-testid="stSidebarCollapsedControl"] svg{
  color:white!important;fill:white!important;}
</style>

```

Продовження додатку А

```

"""', unsafe_allow_html=True)

#
# DATA CLASSES
#
@dataclass
class DeliveryStop:
    address: str
    lat: float
    lon: float
    source: str = "typed"      # "typed" | "map_click"
    node_id: Optional[int] = None
    weight_kg: float = 1.0     # package weight  $q_k$  (kg)
    tw_open: float = 0.0      # time window open  $\delta_{min}$  (seconds from midnight)
    tw_close: float = 86400.0 # time window close  $\delta_{max}$  (seconds from midnight)
    service_time: float = 0.0 # service time  $s_i$  (seconds)

@dataclass
class LegInfo:
    from_label: str
    to_label: str
    distance_m: float
    travel_time_s: float
    cumulative_time_s: float

@dataclass
class ModeResult:
    mode: str
    tsp_route: list
    full_route: list
    total_time_s: float

```

Продовження додатку А

legs: list = field(*default_factory*=list)

For drive (hybrid last-meter): stops in visit order for map markers.

stop_visit_order: Optional[list] = None

ДОДАТОК Б